



A Computation of the 9th Dedekind Number using FPGA Supercomputing

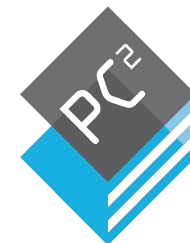
Lennart Van Hirtum

Patrick De Causmaecker, Jens Goemaere, PC2

Paderborn University, Germany

Paderborn Center for Parallel Computing

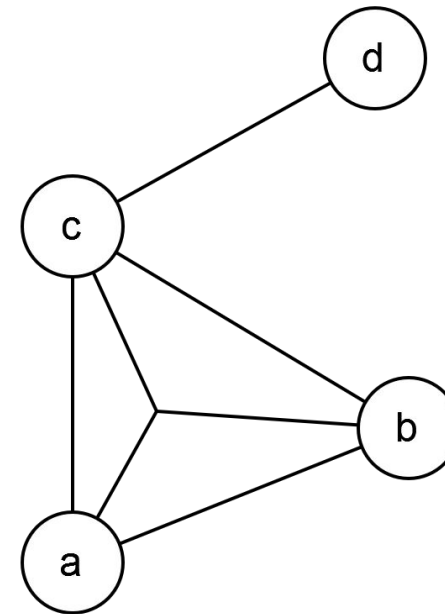
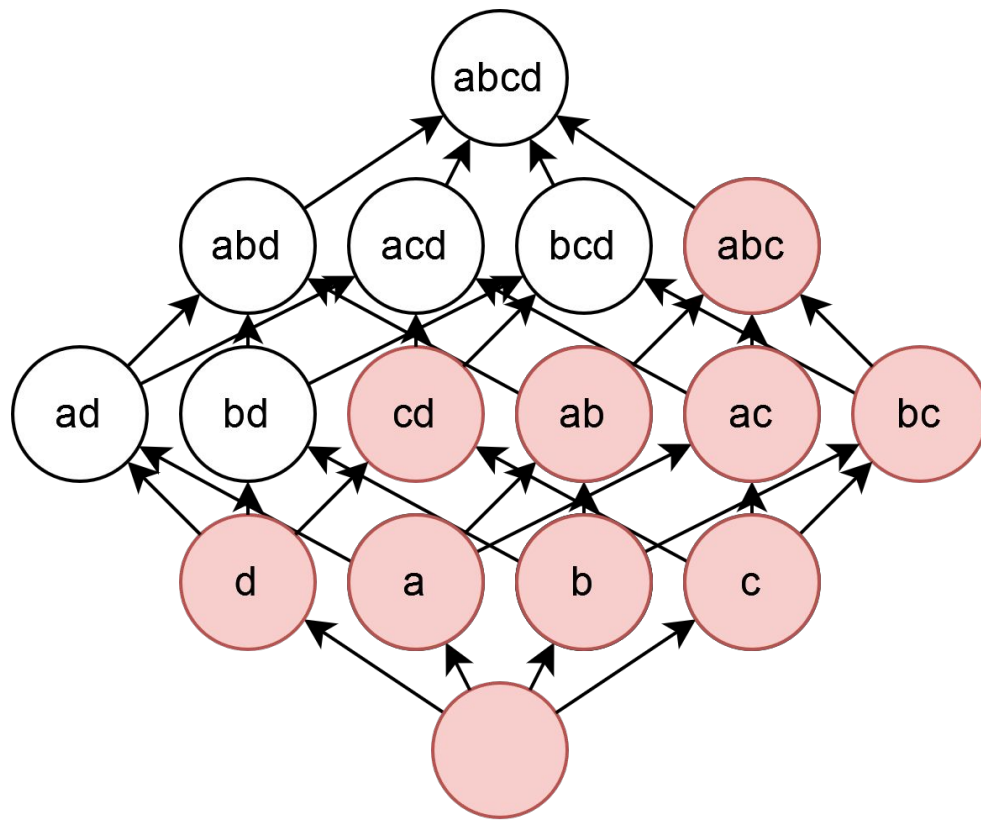
KULAK, 22 May 2023



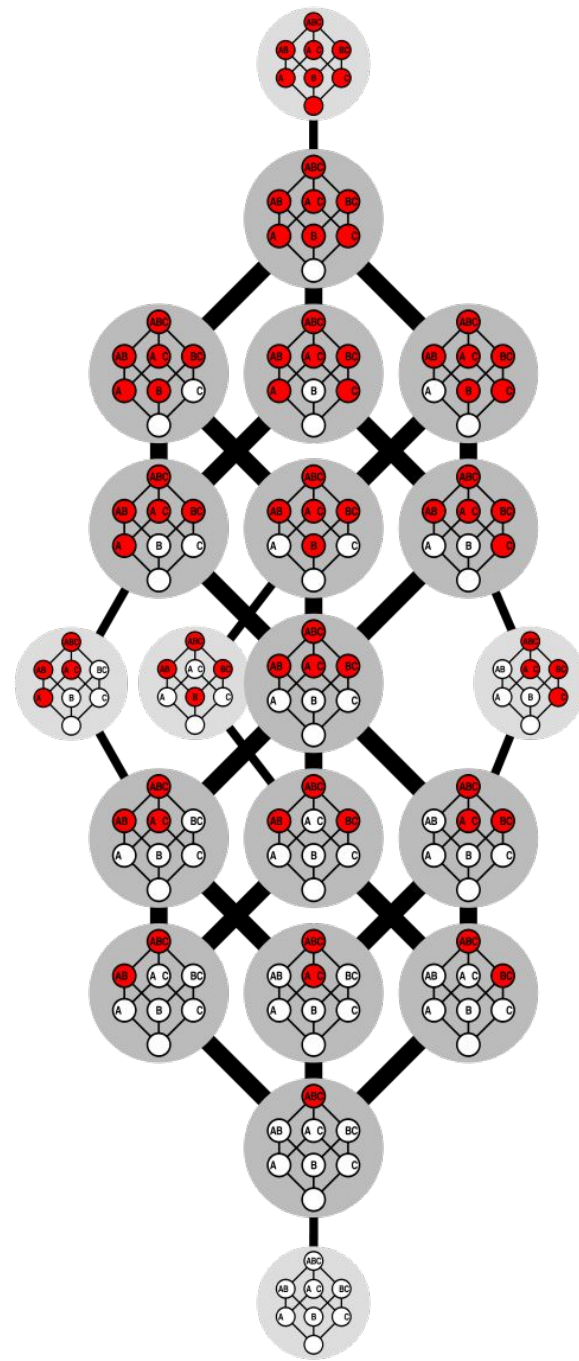
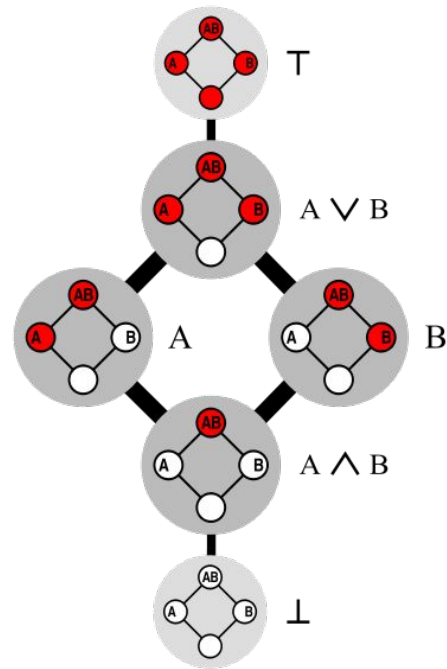
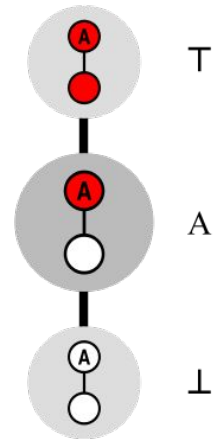
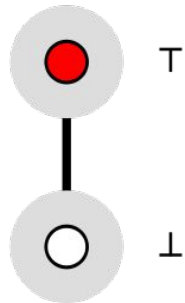
Paderborn
Center for
Parallel
Computing

D(0) =	2	Dedekind (1897)
D(1) =	3	Dedekind (1897)
D(2) =	6	Dedekind (1897)
D(3) =	20	Dedekind (1897)
D(4) =	168	Dedekind (1897)
D(5) =	7581	Church (1940)
D(6) =	7828354	Ward (1946)
D(7) =	2414682040998	Church (1965)
D(8) =	56130437228687557907788	Wiedemann (1991)
D(9) =	286386577668298411128469151667598498812366 (2023)	

Monotone Boolean Functions



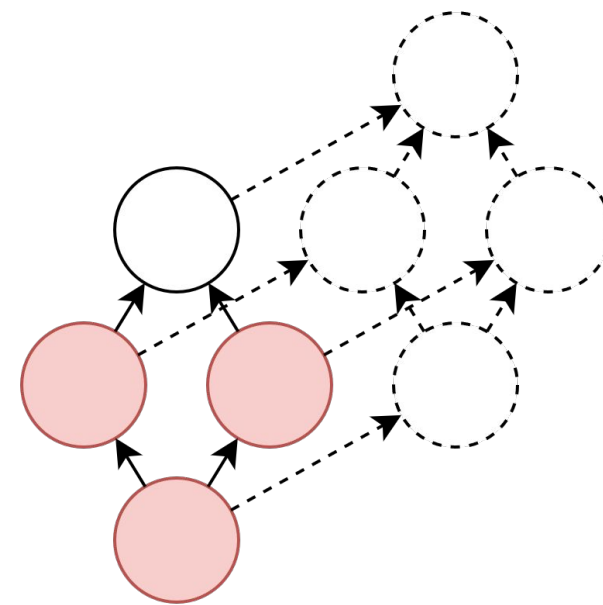
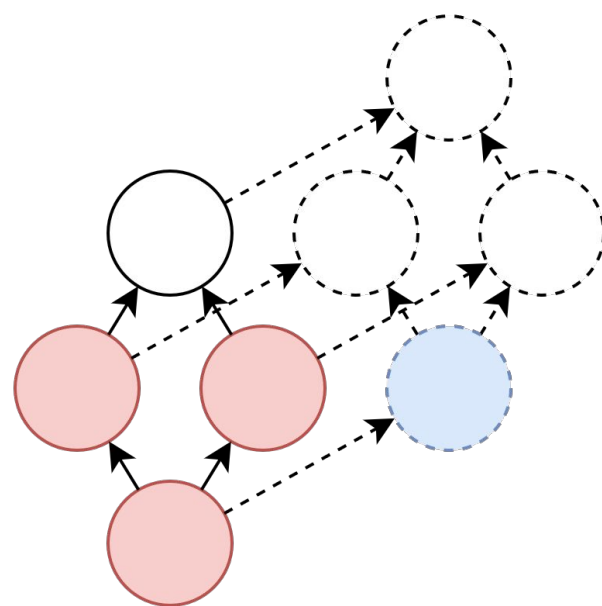
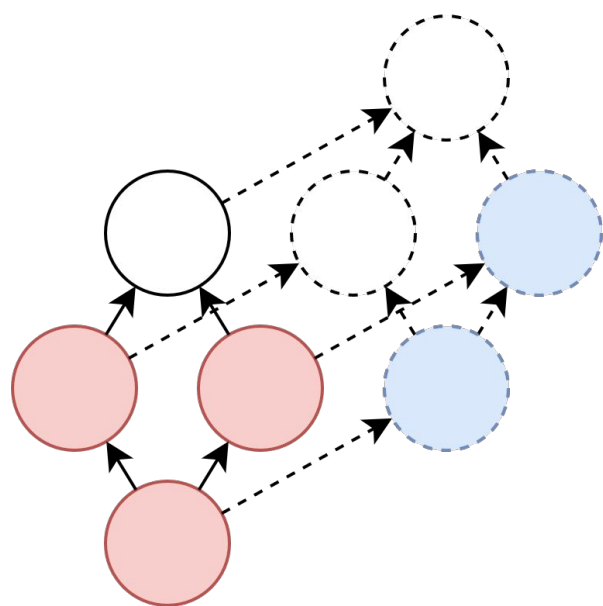
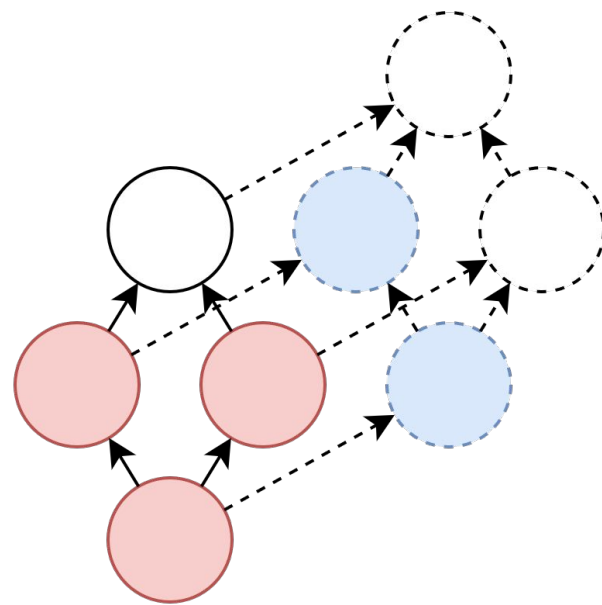
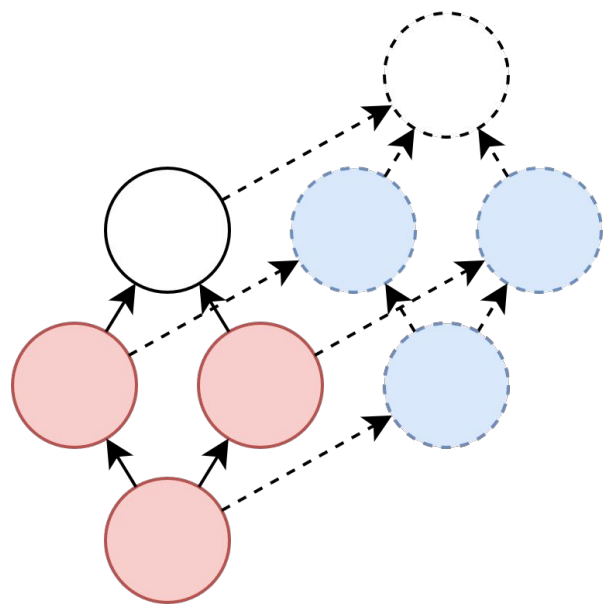
abcd	bcd	acd	cd	abd	bd	ad	d	abc	bc	ac	c	ab	b	a	
------	-----	-----	----	-----	----	----	---	-----	----	----	---	----	---	---	--



Jumping Formulas

$$D(n+1) = \sum_{\alpha \in A_n} |[\perp, \alpha]|$$

Core Idea

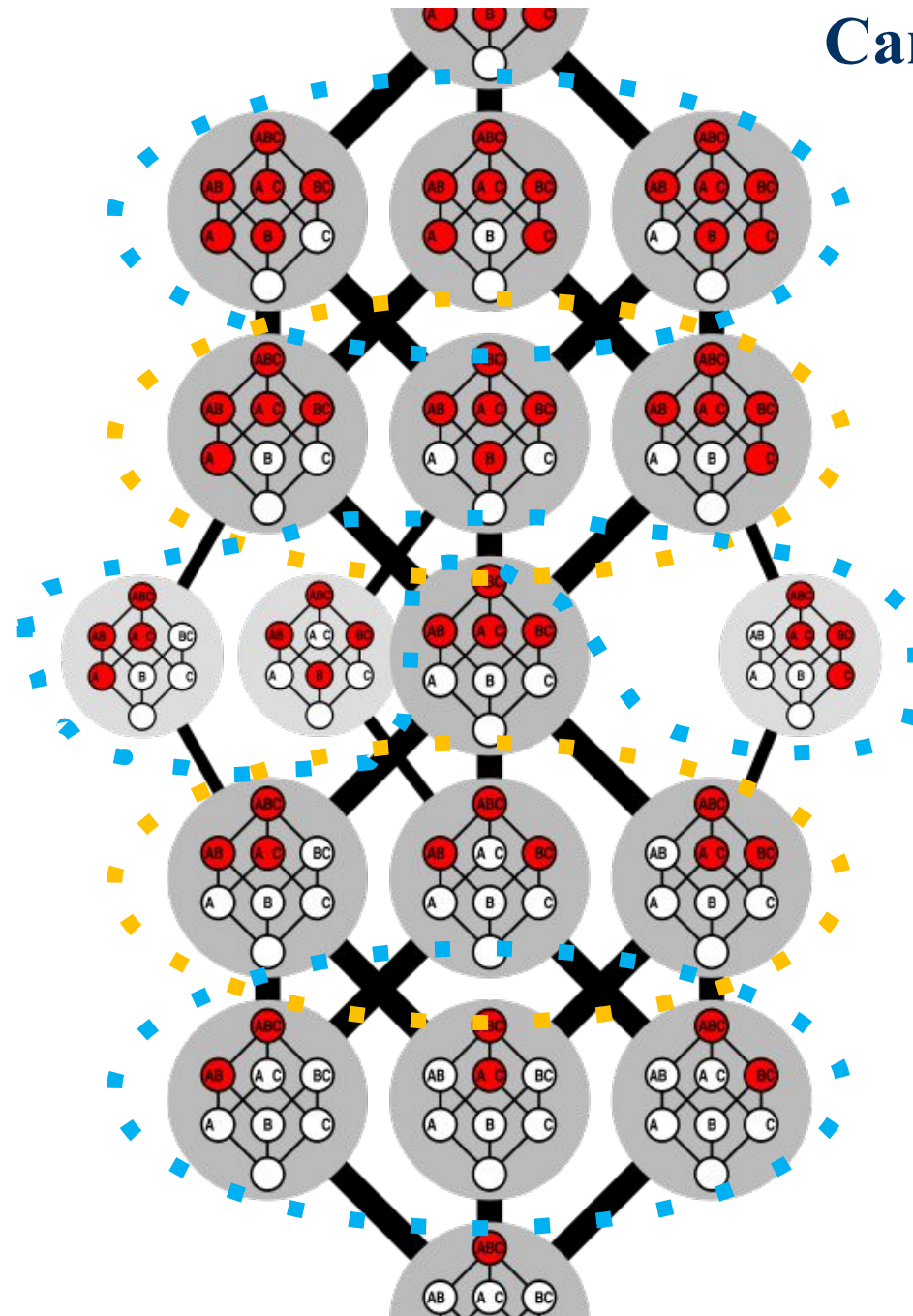
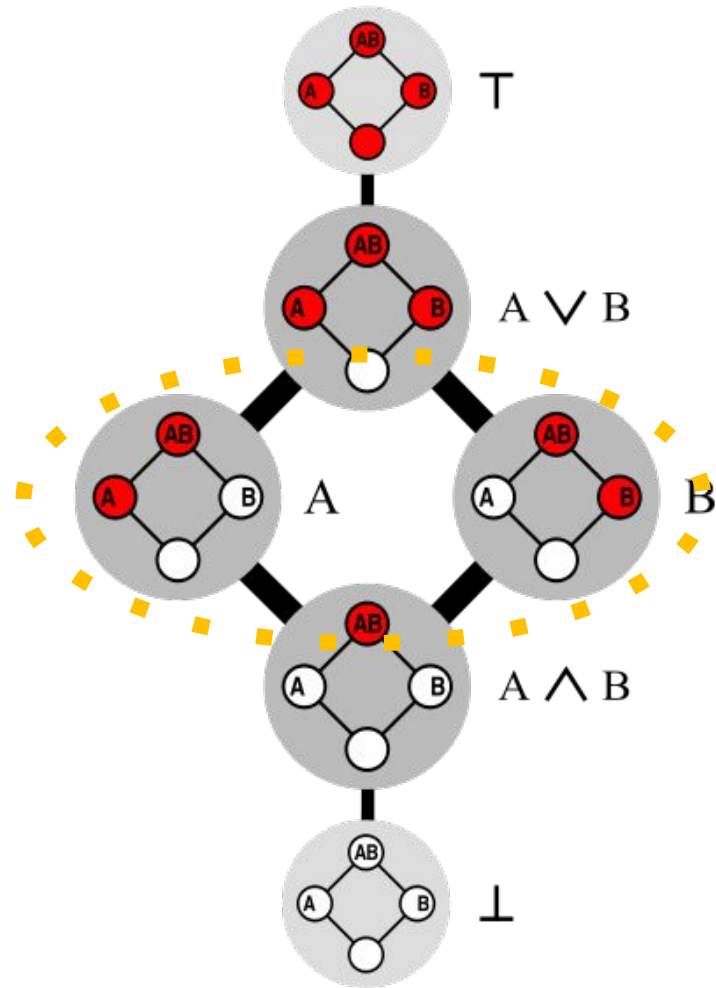


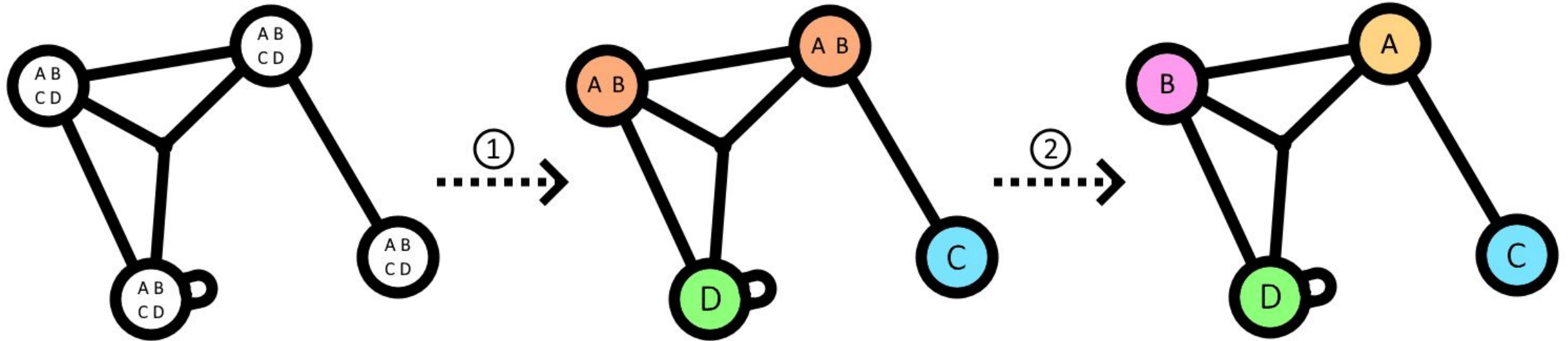
$$D(n+1) = \sum_{\alpha \in A_n} |[\perp, \alpha]|$$



$$D(n+1) = \sum_{\alpha \in R_n} D_\alpha |[\perp, \alpha]|$$

Canonization

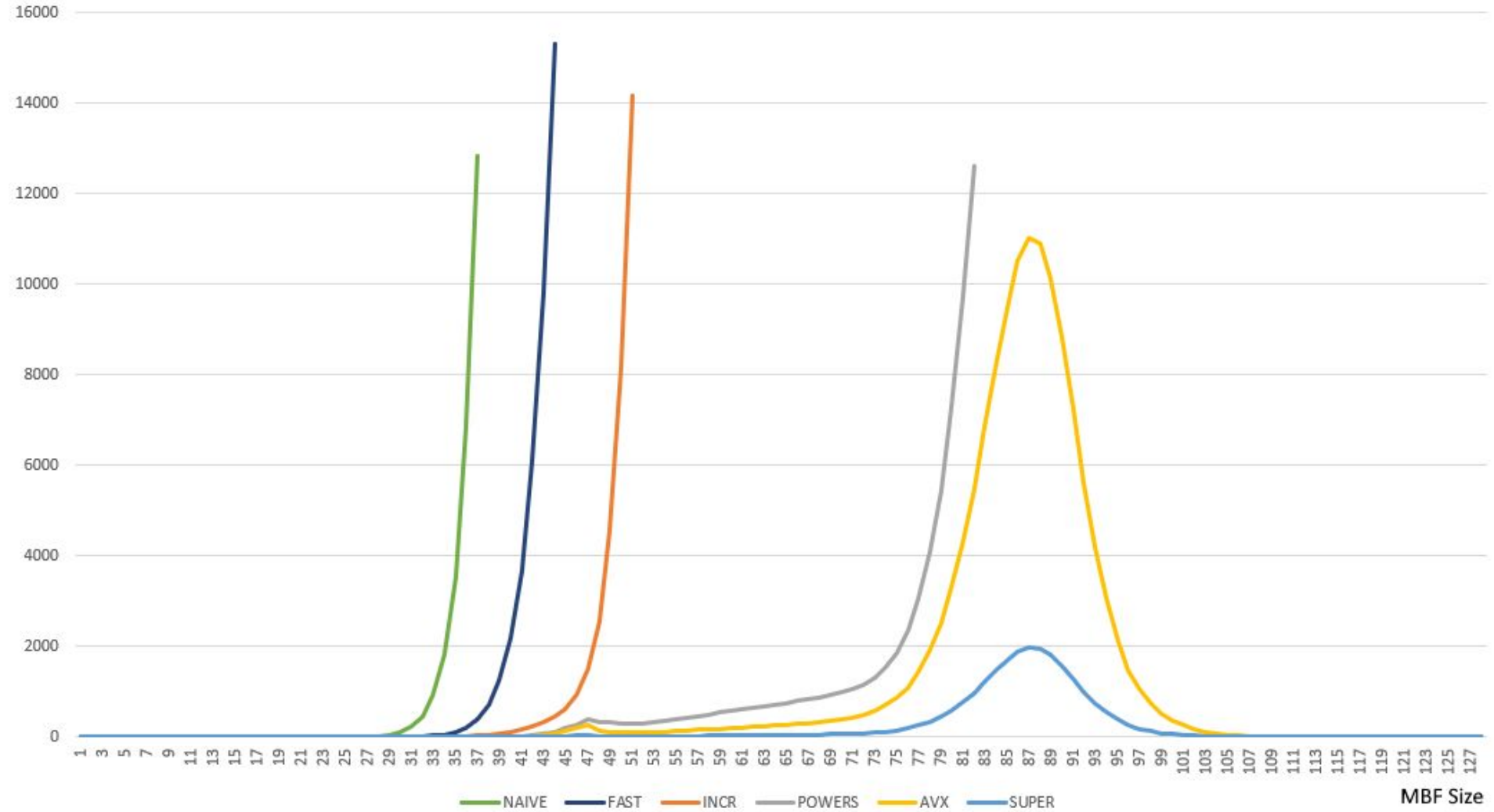




R(0) =	2	
R(1) =	3	
R(2) =	5	
R(3) =	10	
R(4) =	30	
R(5) =	210	
R(6) =	16353	
R(7) =	490013148	Yusan (2012)
R(8) =	1392195548889993358	Pawelski (2021)
R(9) =	789204635842035040527740846300252680 (Paw 2023)	

$$D(n+2) = \sum_{\substack{\alpha, \beta \in D_n \\ \alpha \leq \beta}} |[\perp, \alpha]| 2^{C_{\alpha, \beta}} |[\beta, \top]|$$

seconds to process all



Expanded P-Coëfficient Formula

$$D(n+2) = \sum_{\substack{\alpha, \beta \in D_n \\ \alpha \leq \beta}} |[\perp, \alpha]| 2^{C_{\alpha, \beta}} |[\beta, \top]|$$


$$D(n+2) = \sum_{\alpha \in R_n} |[\perp, \alpha]| D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} |[\beta, \top]| \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

$$|[\perp, \alpha]| 2^{C_{\alpha, \beta}} |[\beta, \top]| = |[\overline{\alpha}, \top]| 2^{C_{\overline{\beta}, \overline{\alpha}}} |[\perp, \overline{\beta}]|$$

Expanded P-Coëfficient Formula

$$D(n+2) = \sum_{\alpha \in R_n} |[\perp, \alpha]| D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} |[\beta, \top]| \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

490M

*45

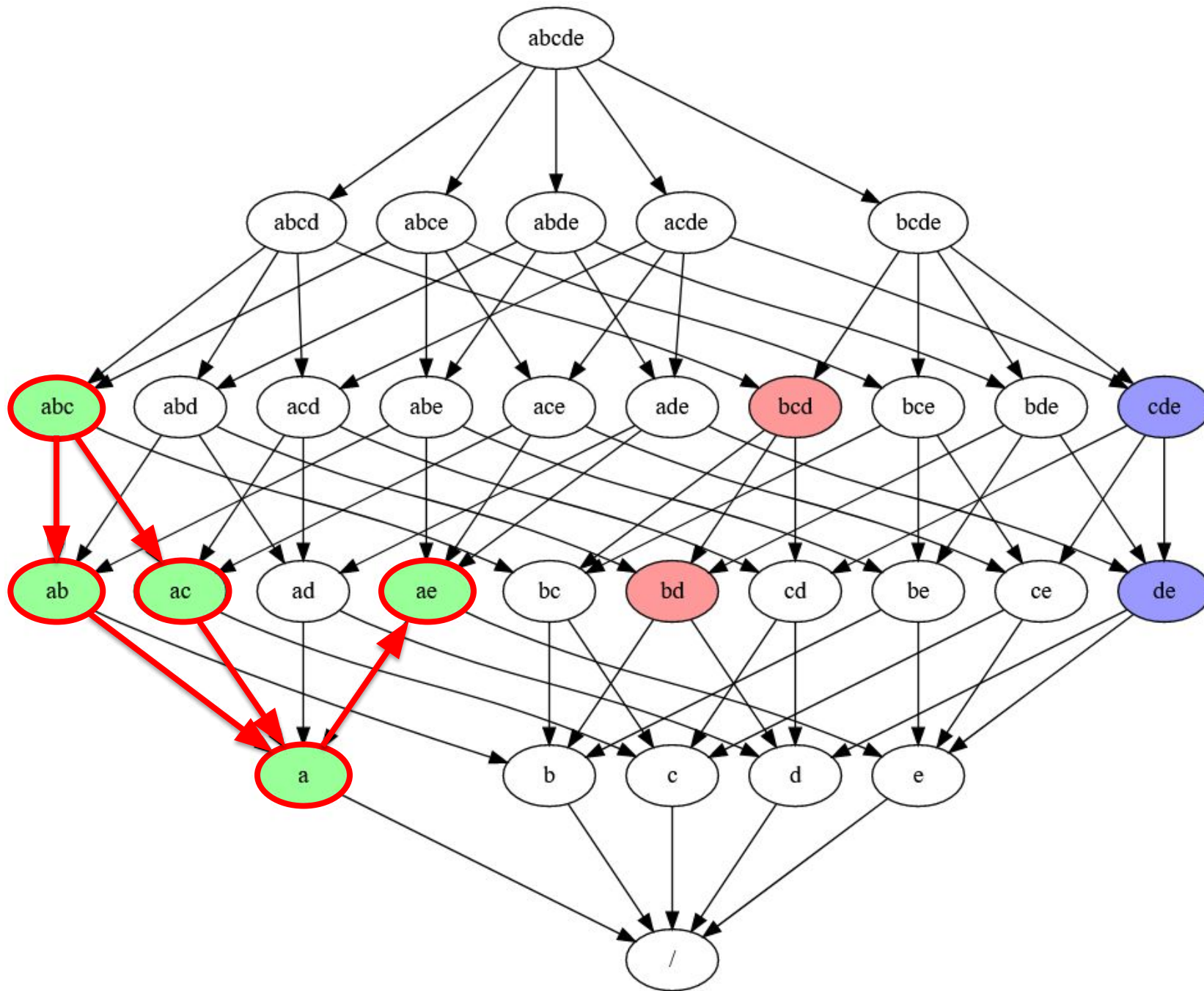
M

*5040

$5.57 * 10^{18}$ $C_{\alpha, \gamma}$ values in total!

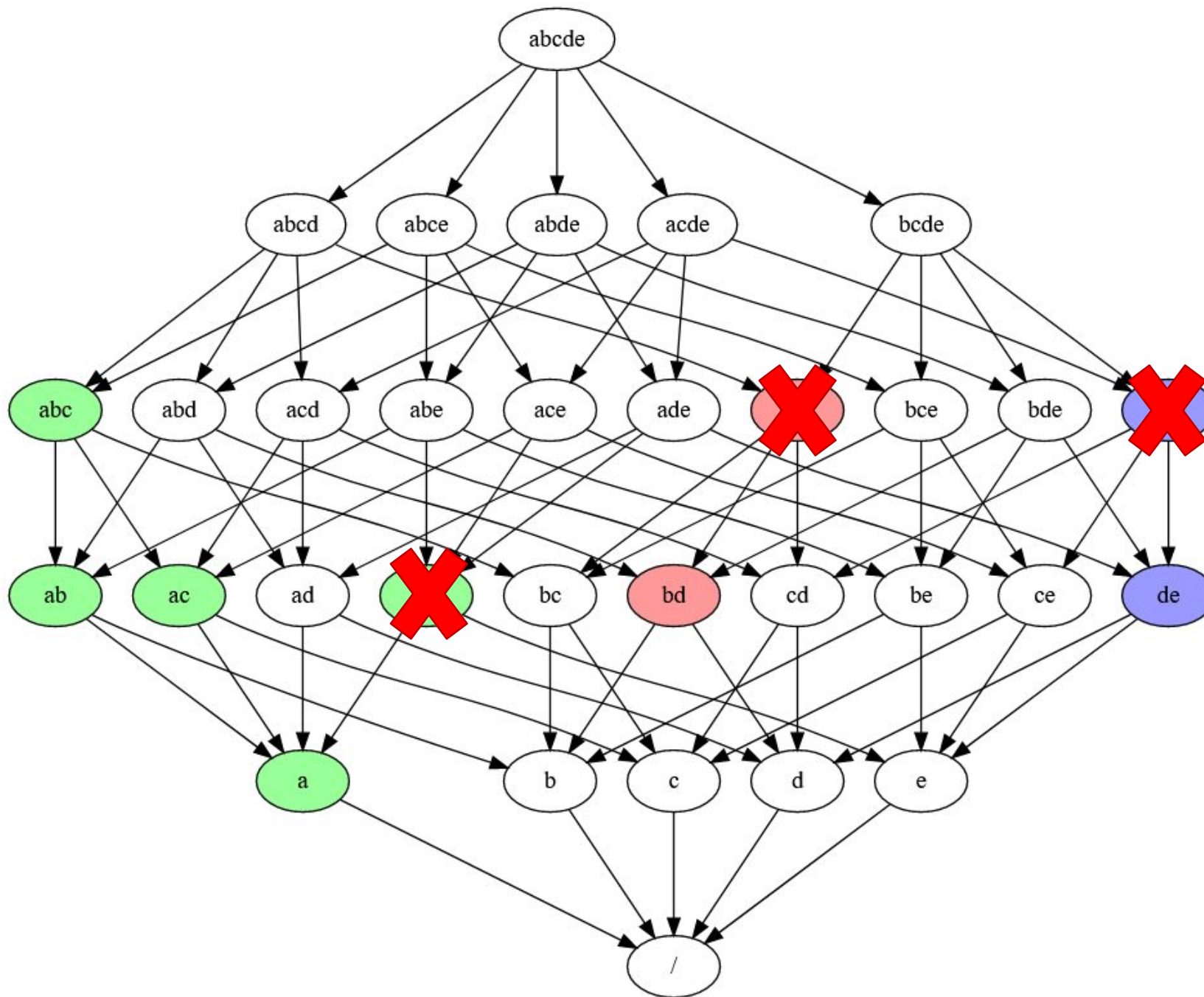
FloodFill Algorithm

FloodFill Algorithm



$$C_{\alpha, \beta} = 3$$

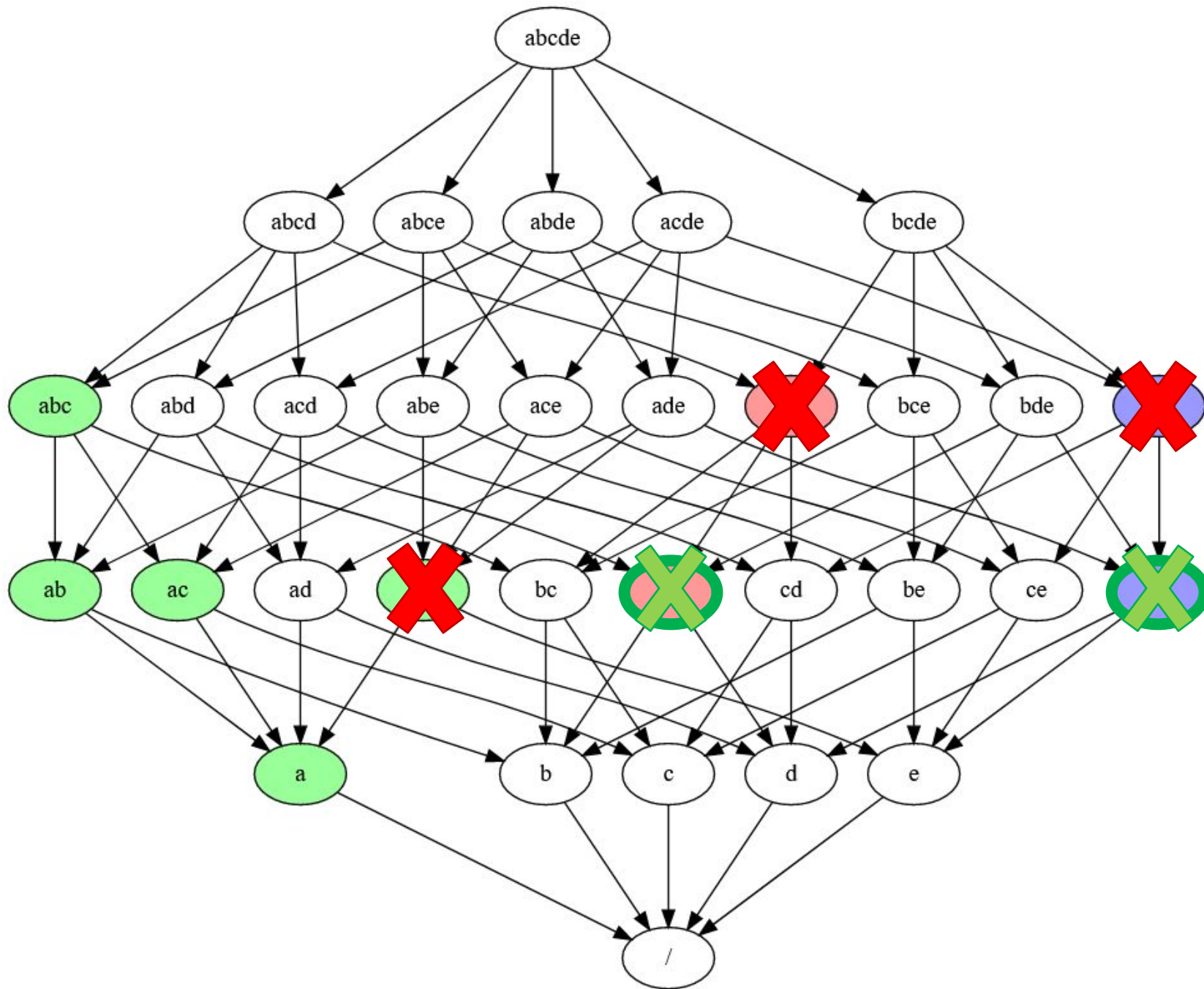
Leaf Elimination



- LE Up

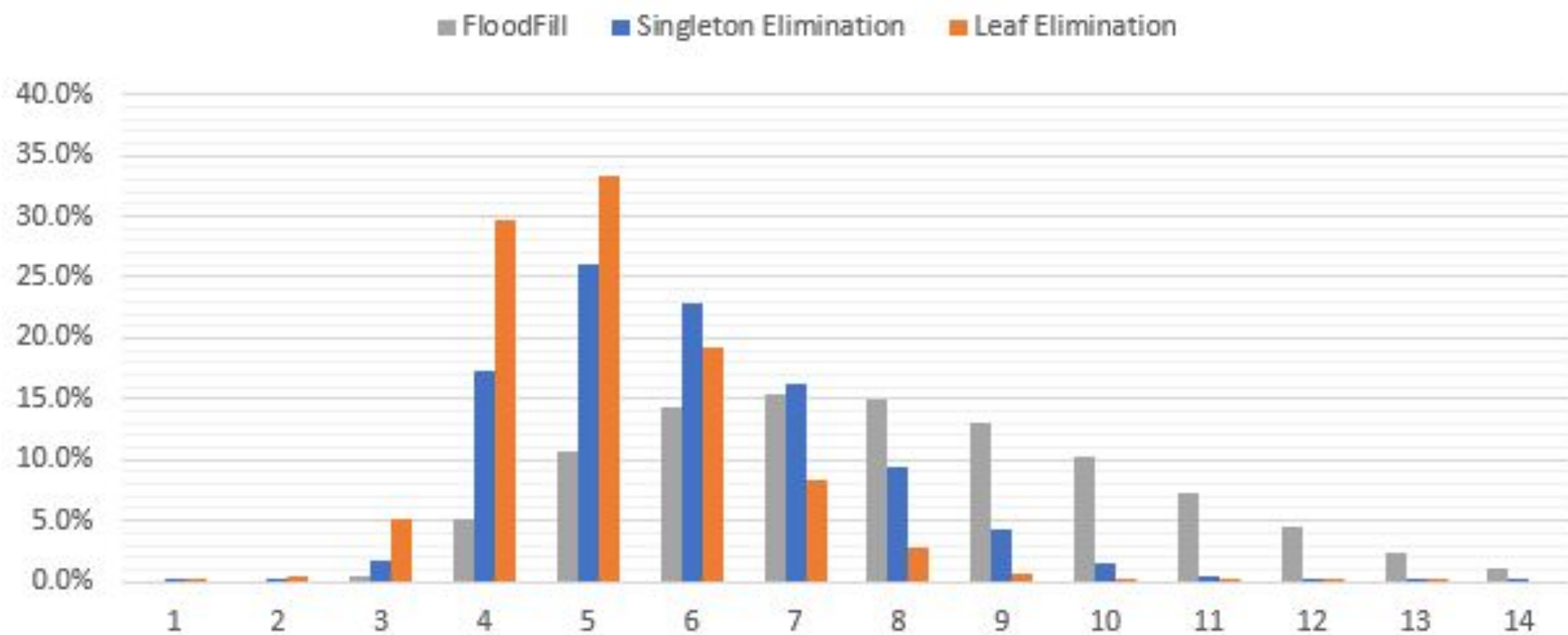
- LE
Down

Singleton Elimination



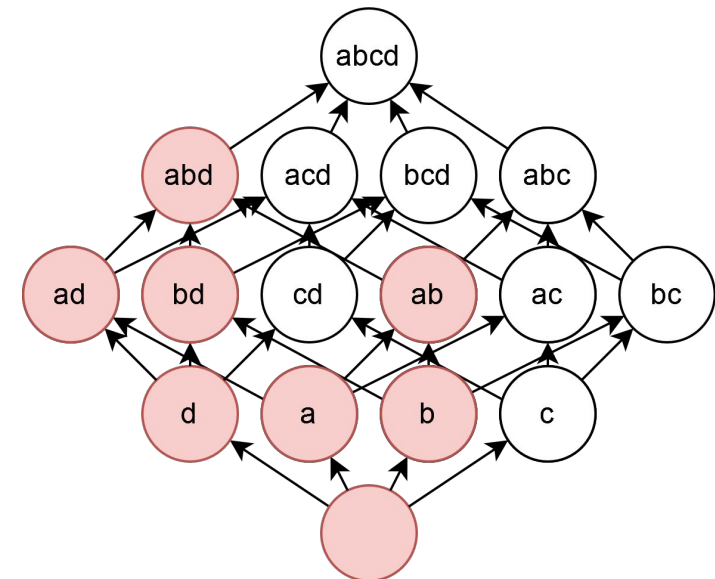
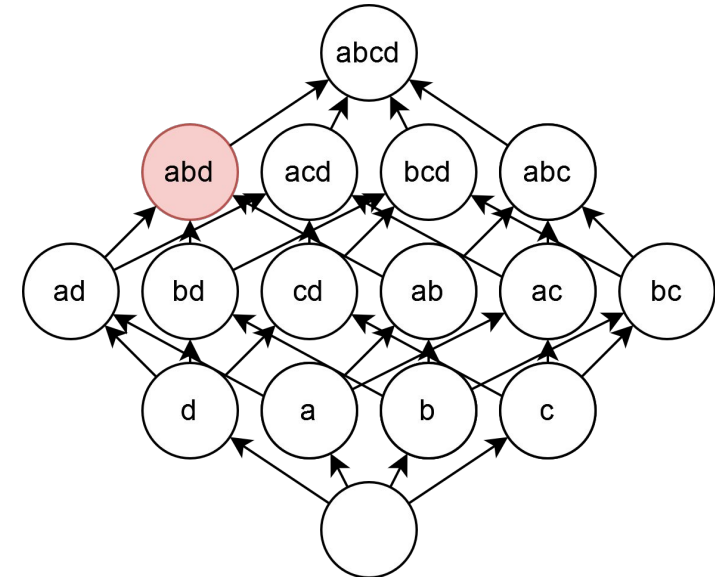
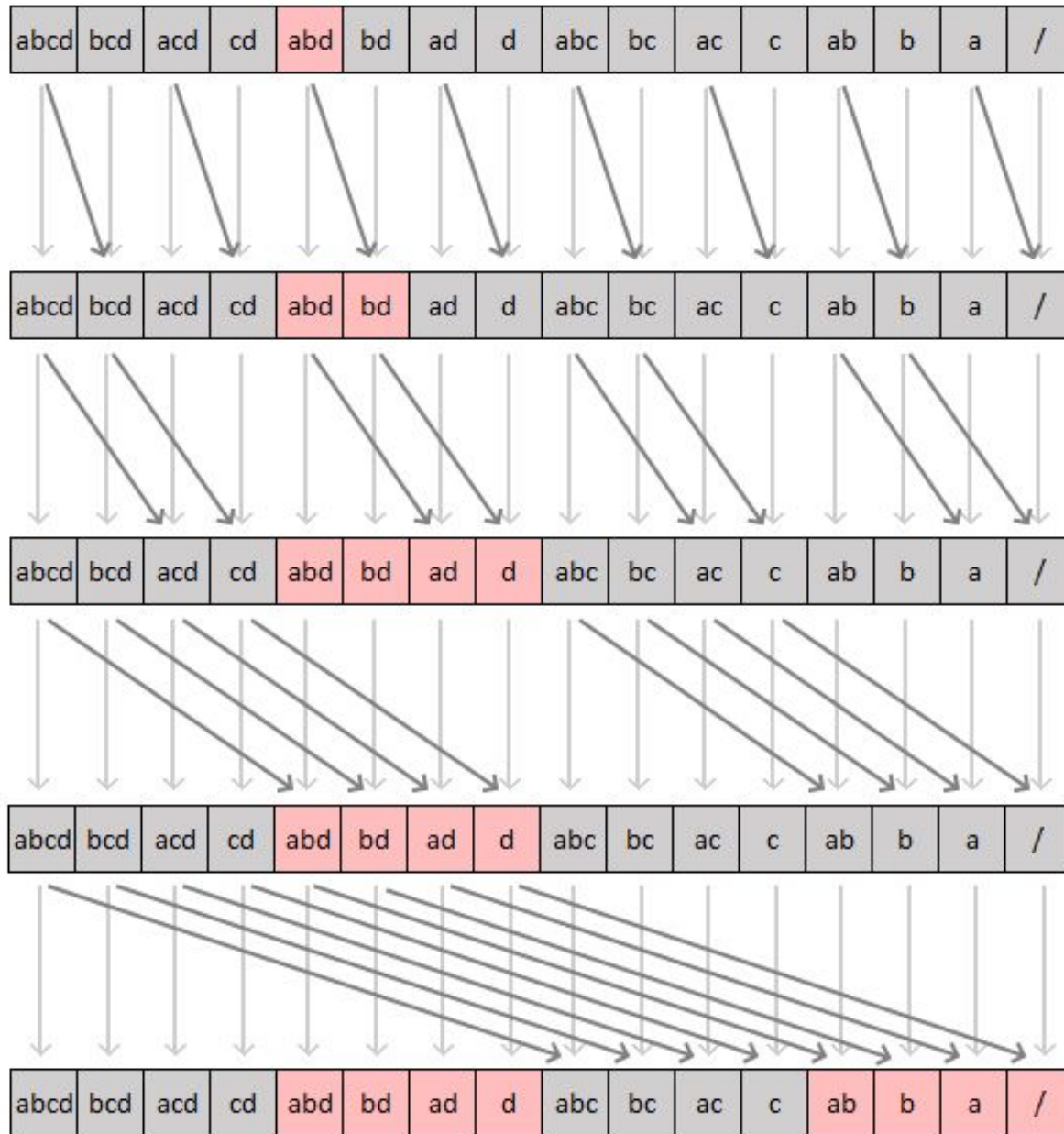
def countConnected(**MBF** α , **MBF** γ): **FloodFill Algorithm**
 BF graph = $\alpha \& \neg \gamma$
 graph = eliminateLeafesUp(graph)
 graph, **int** count = eliminateSingletons(graph)
 while graph **not** empty:
 BF seed = firstNode(graph)
 do:
 BF seedUp = monotonizeUp(seed) & graph
 seed = monotonizeDown(seedUp) & graph
 while seedUp != seed
 graph = graph & $\neg$ seed
 count++

FloodFill Cycles Distribution

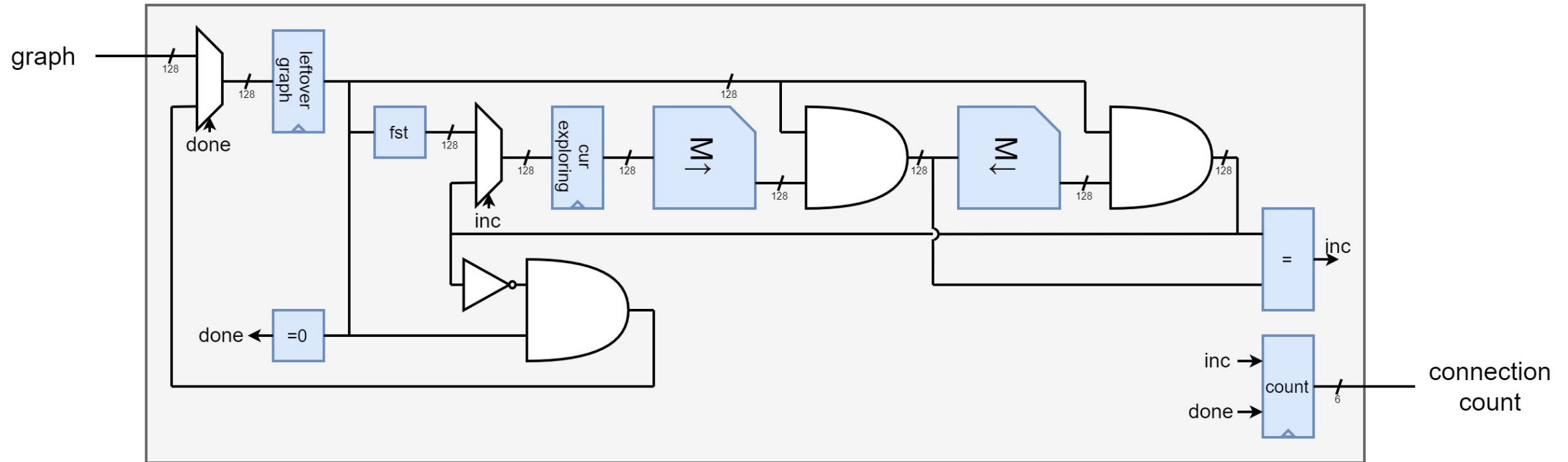


Hardware Implementation

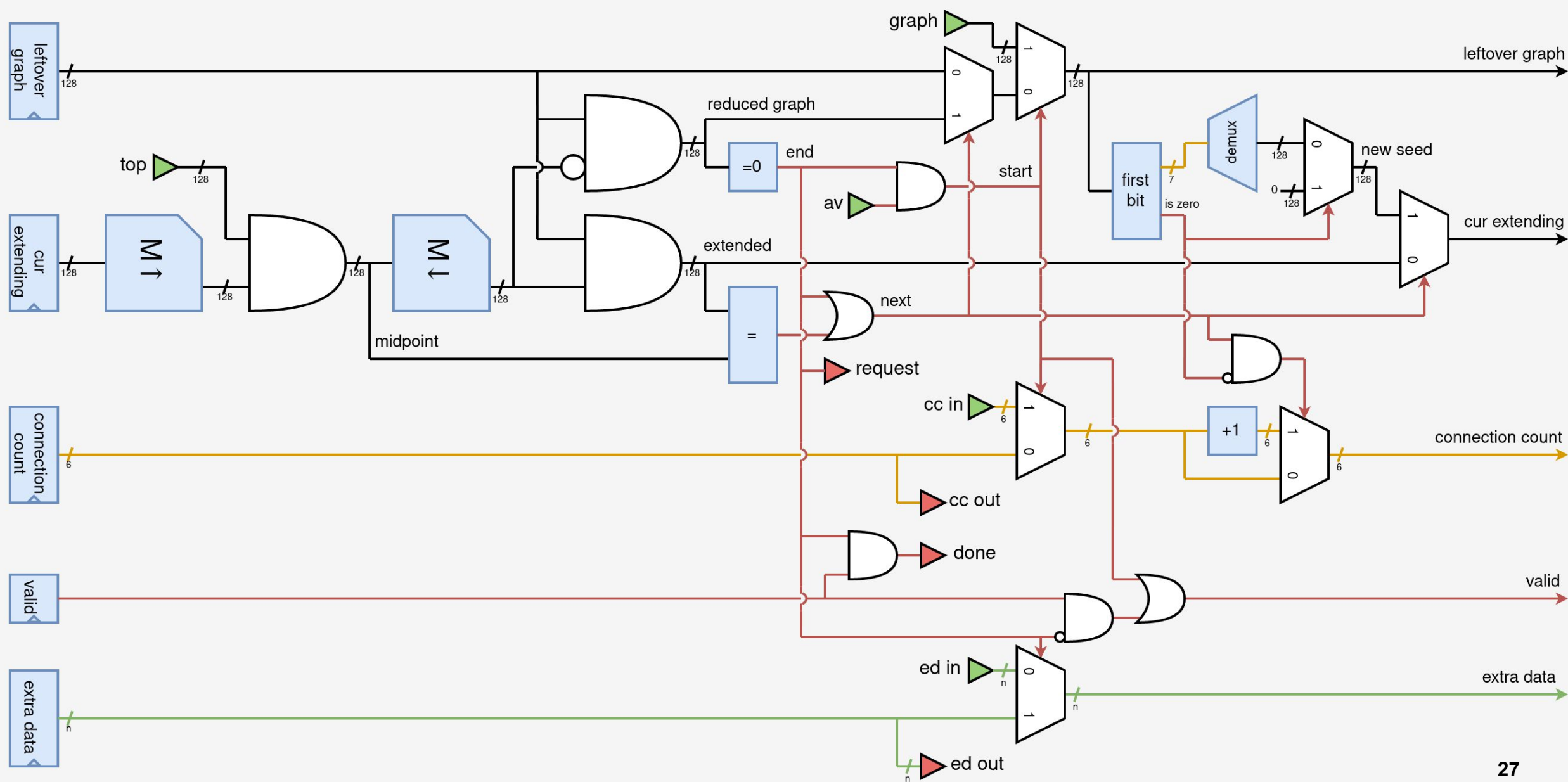
Monotonization



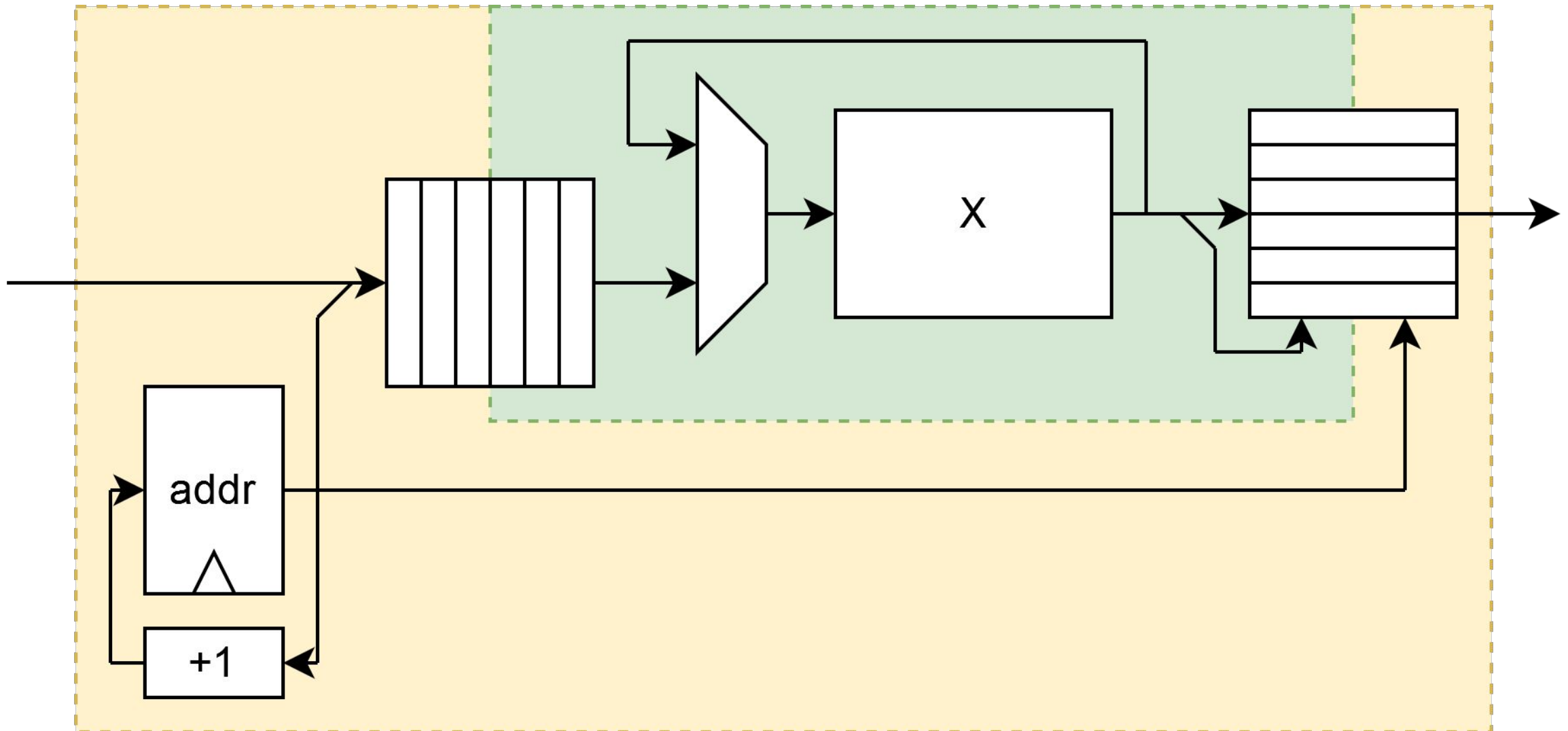
Count Connected Core



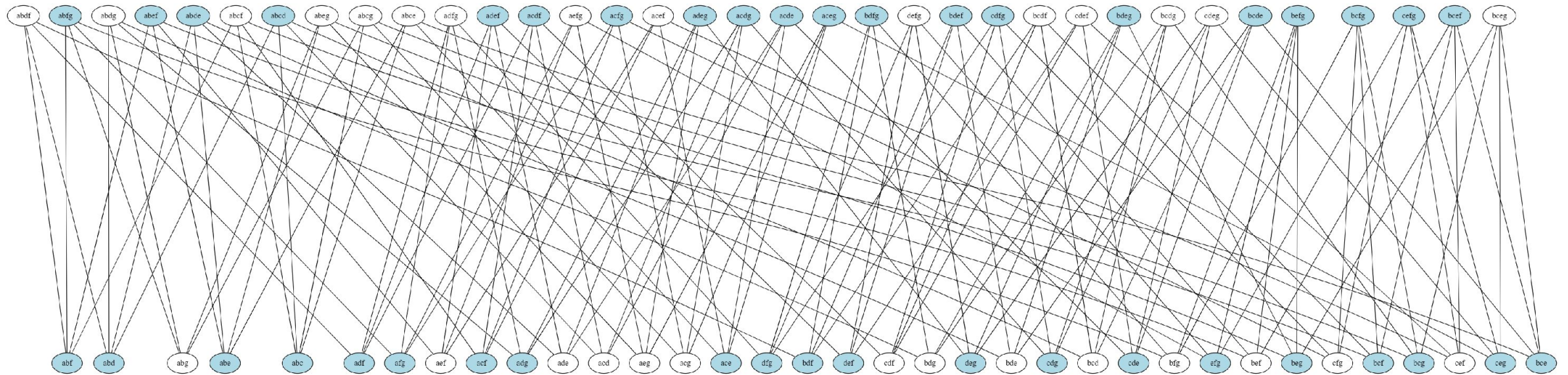
Pipelined



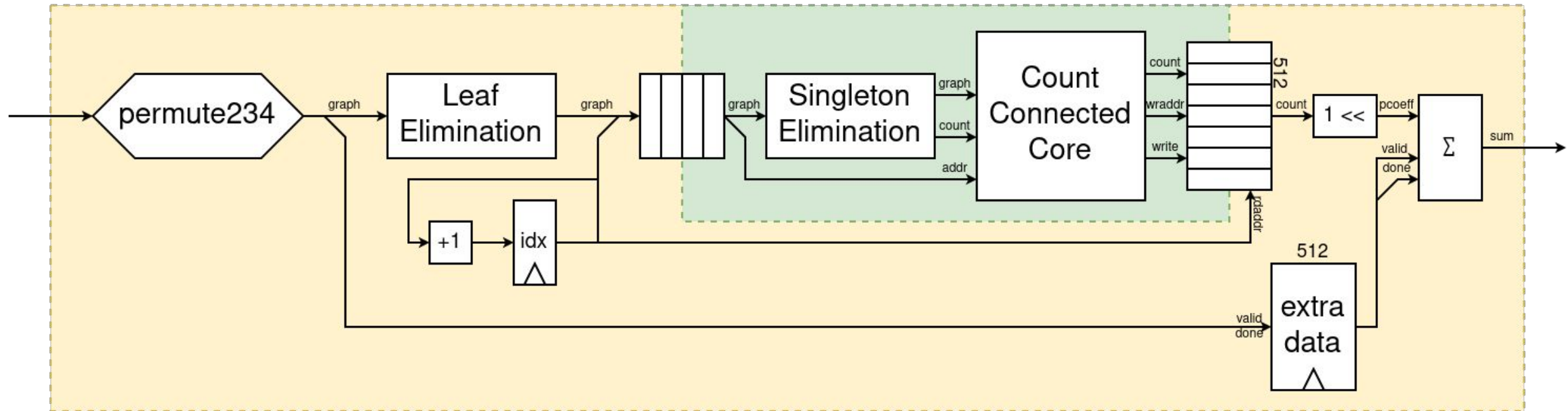
Loop implementation

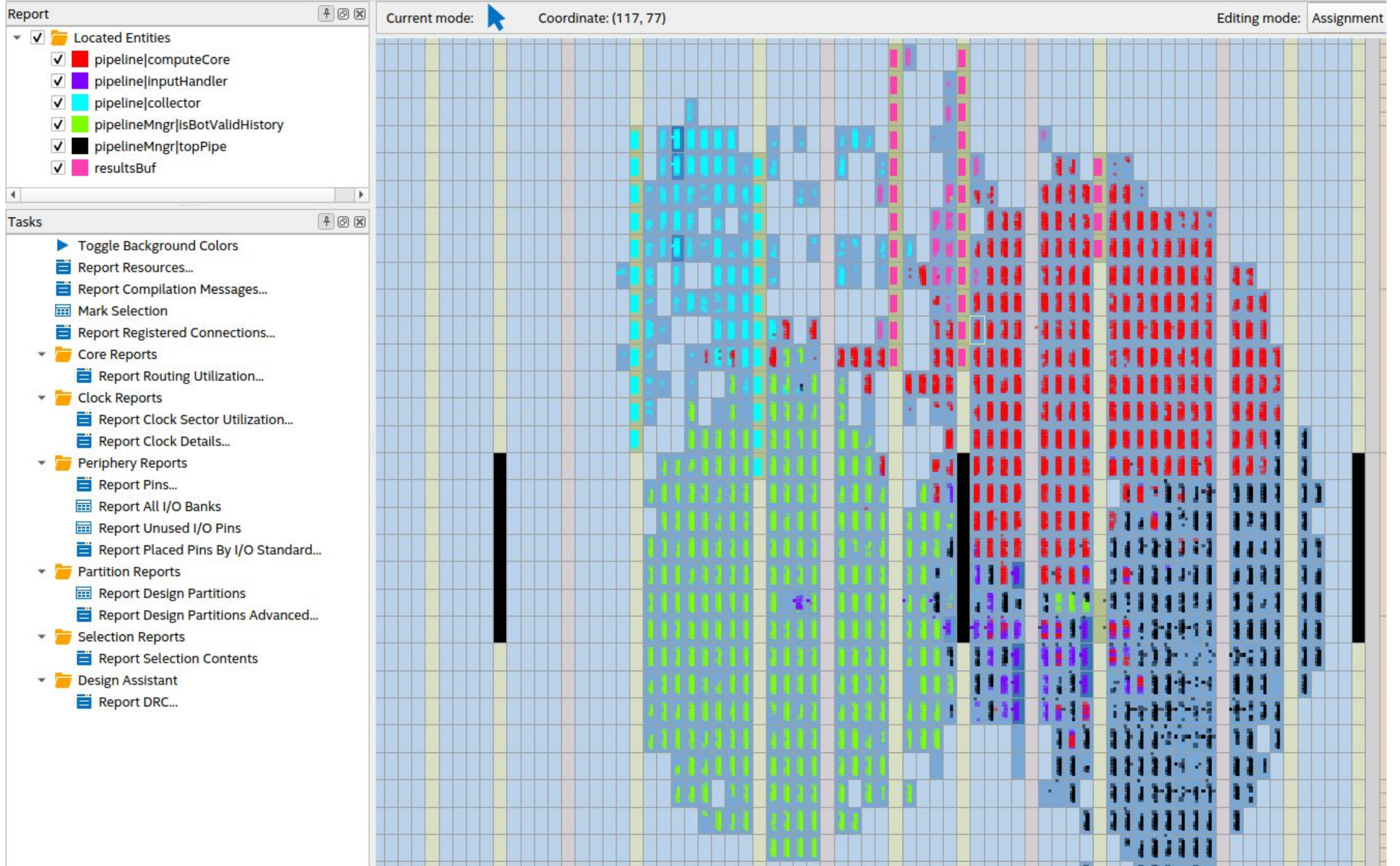


Worst Case

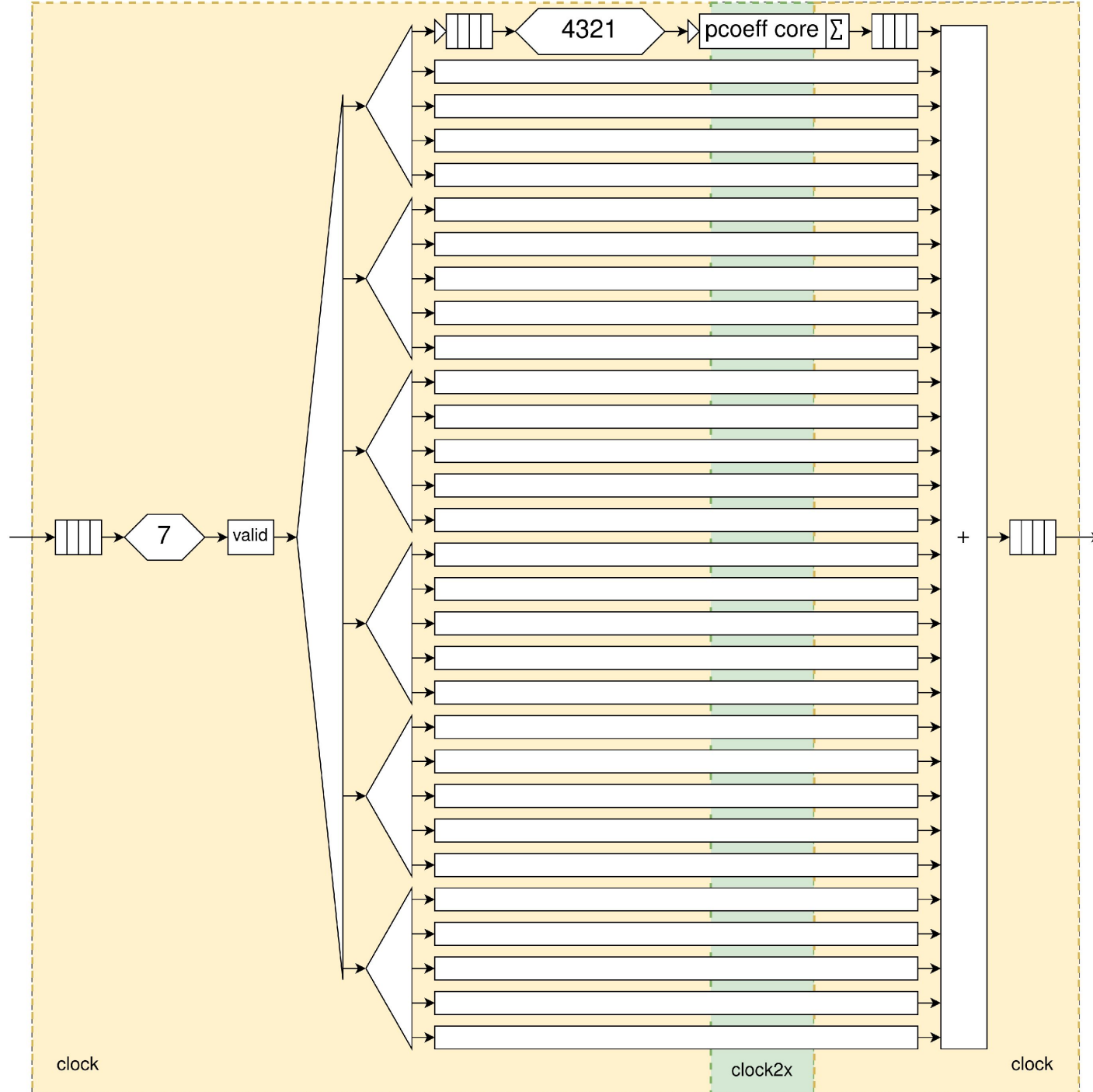


Processing Module



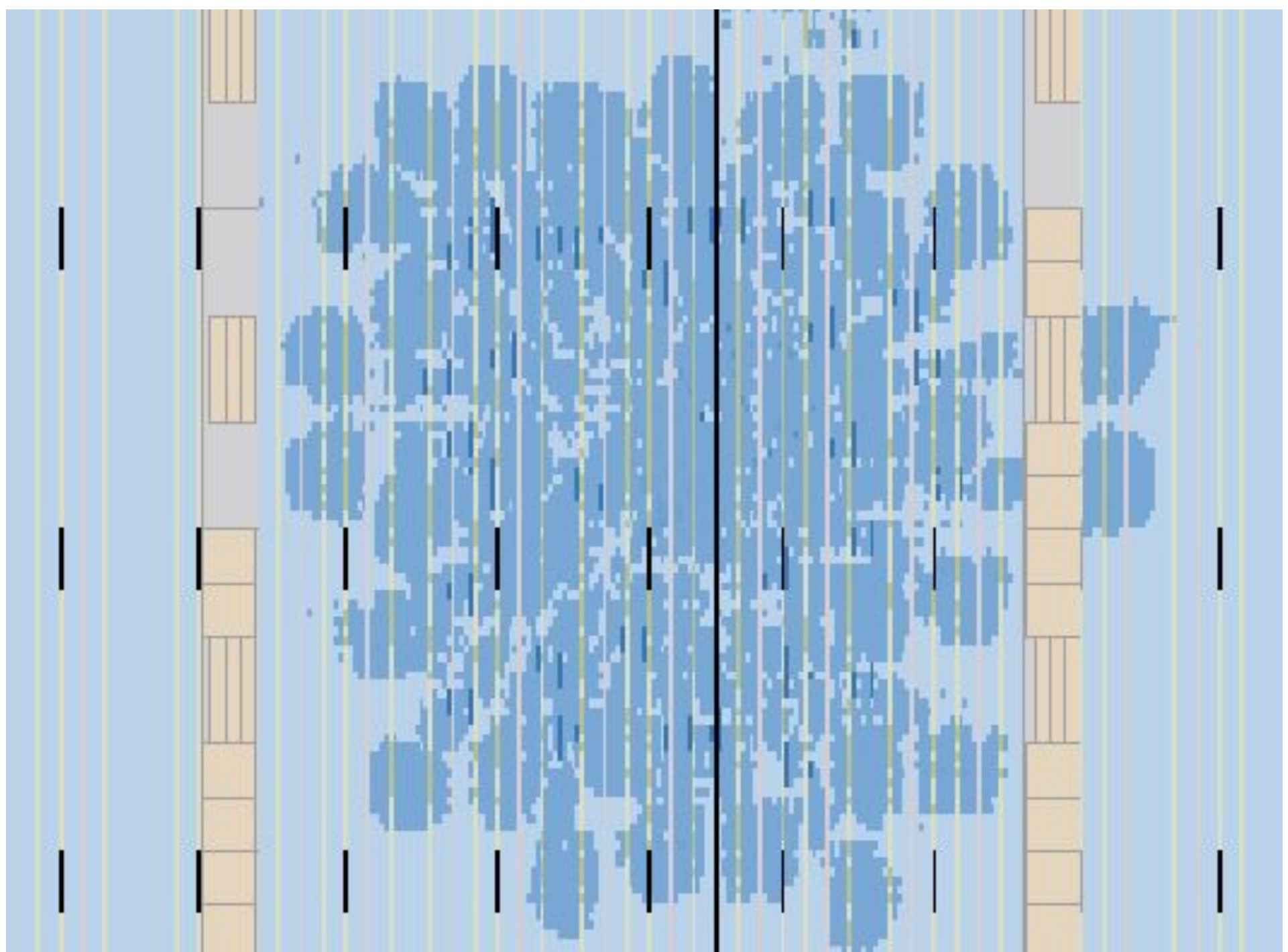


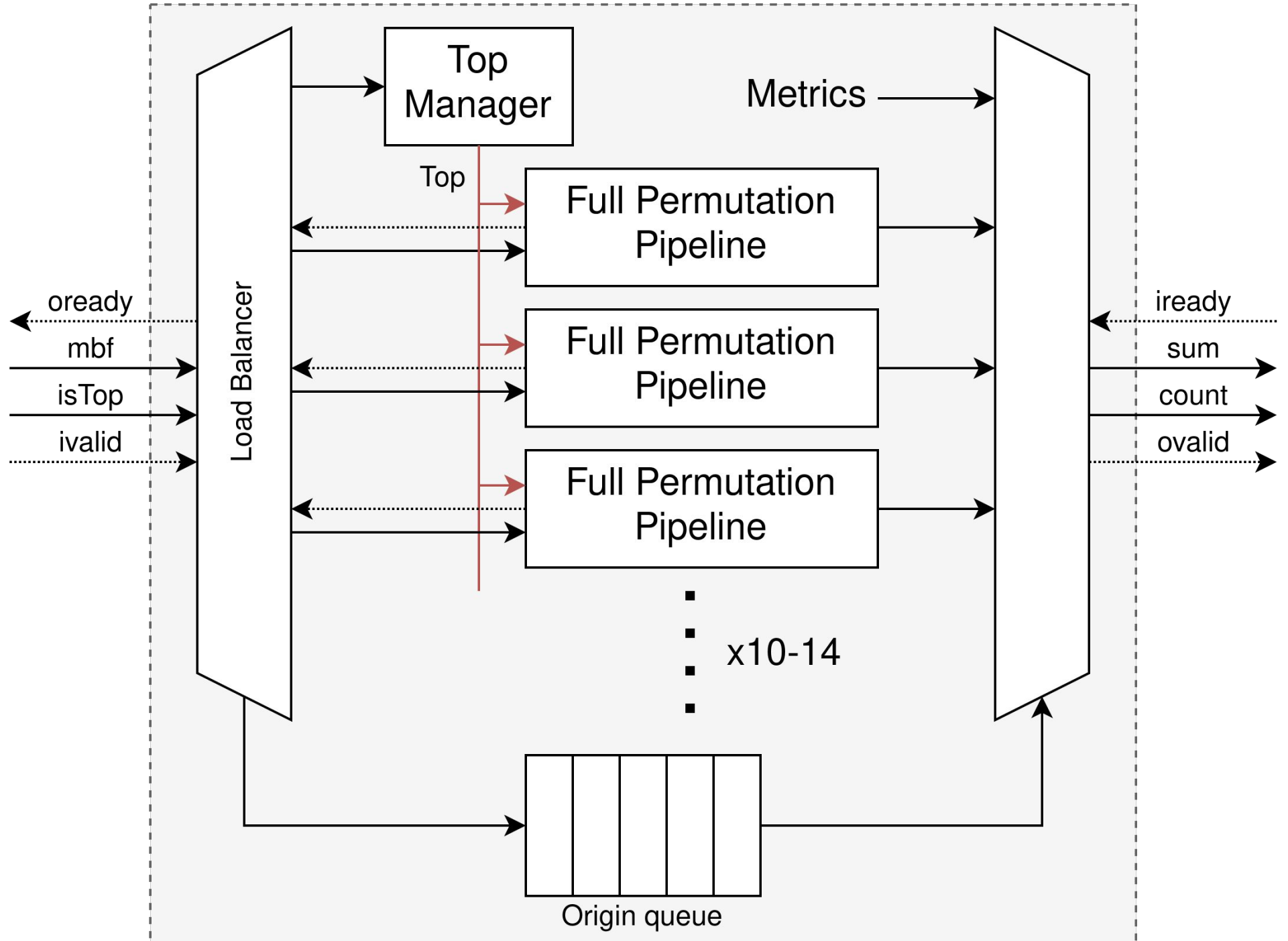
5040 Permutation splitter

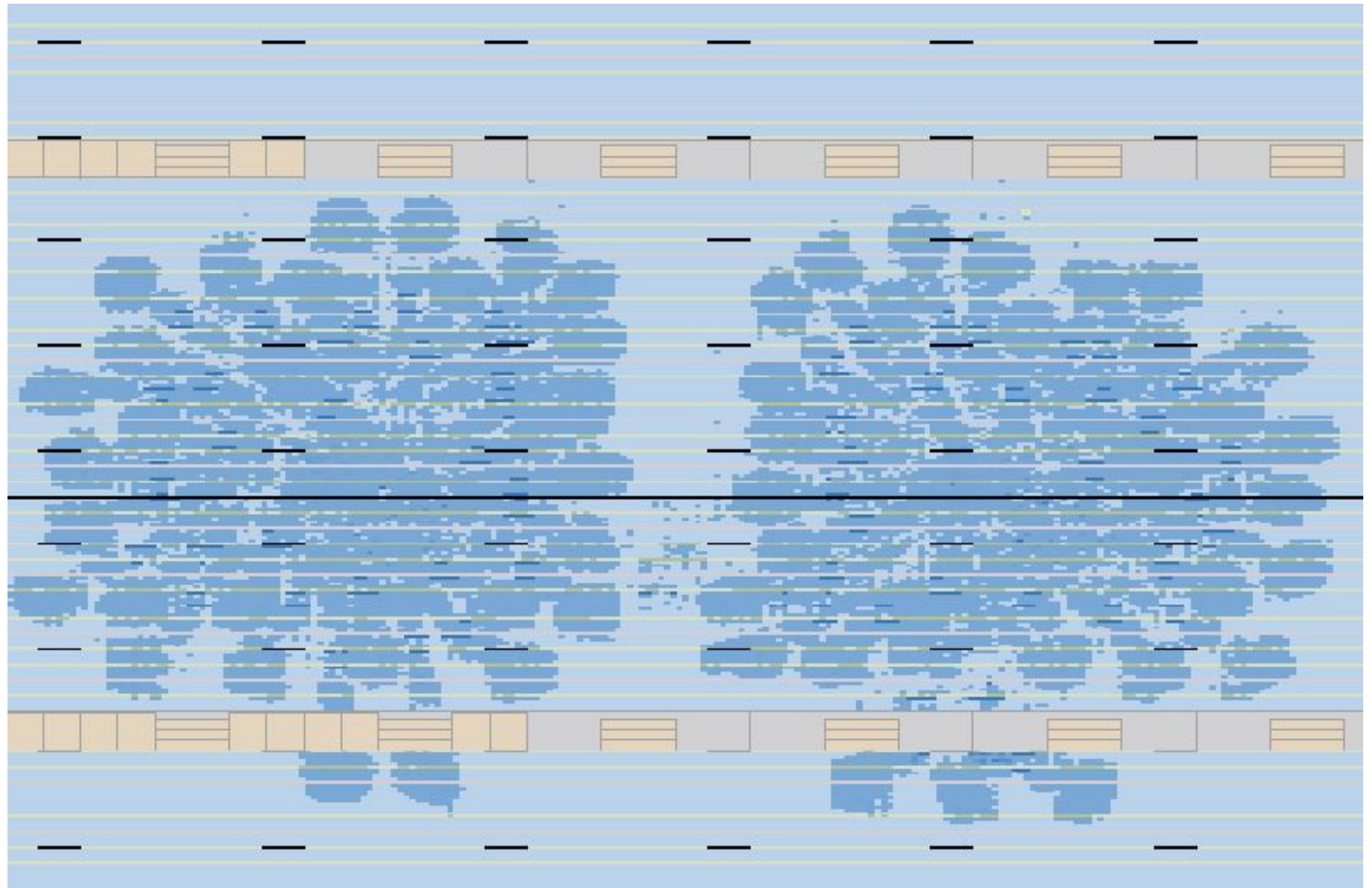


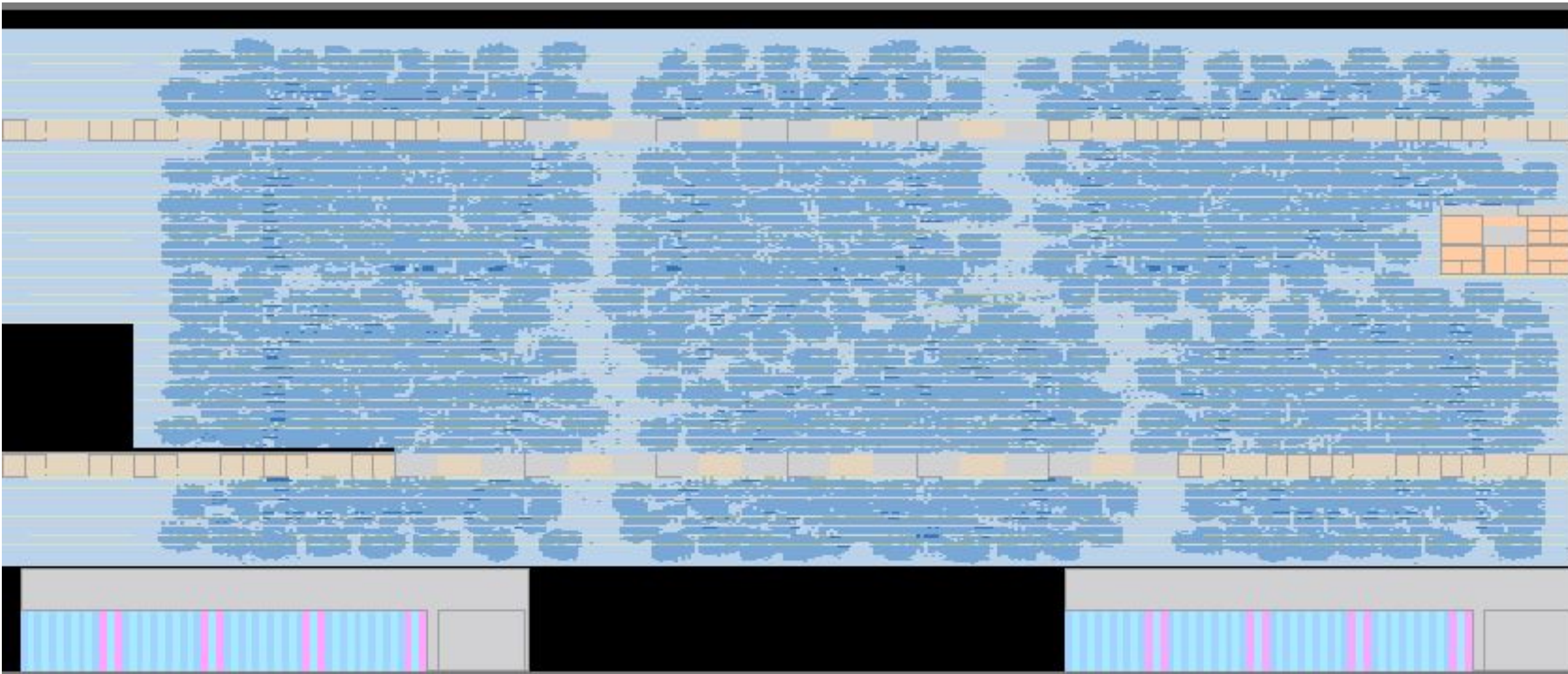
$$\sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

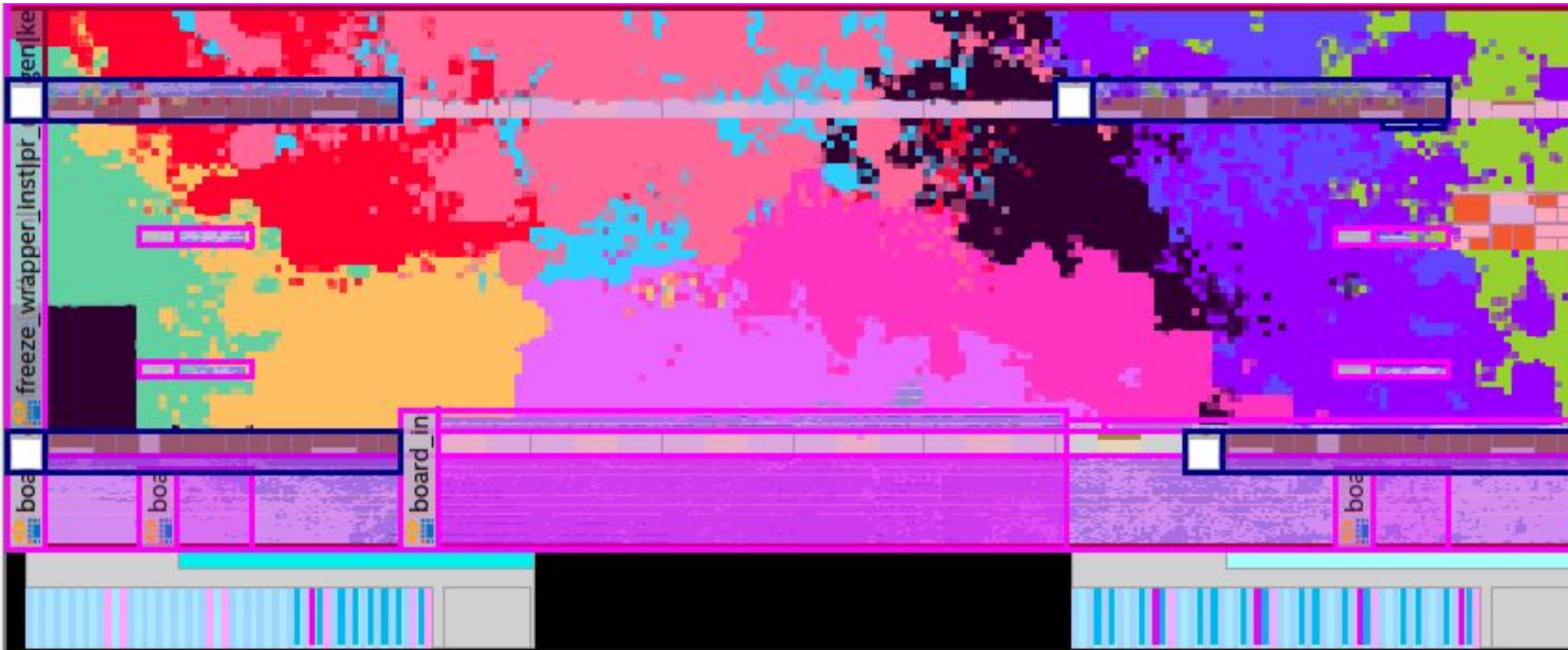
7! = 5040
permutations





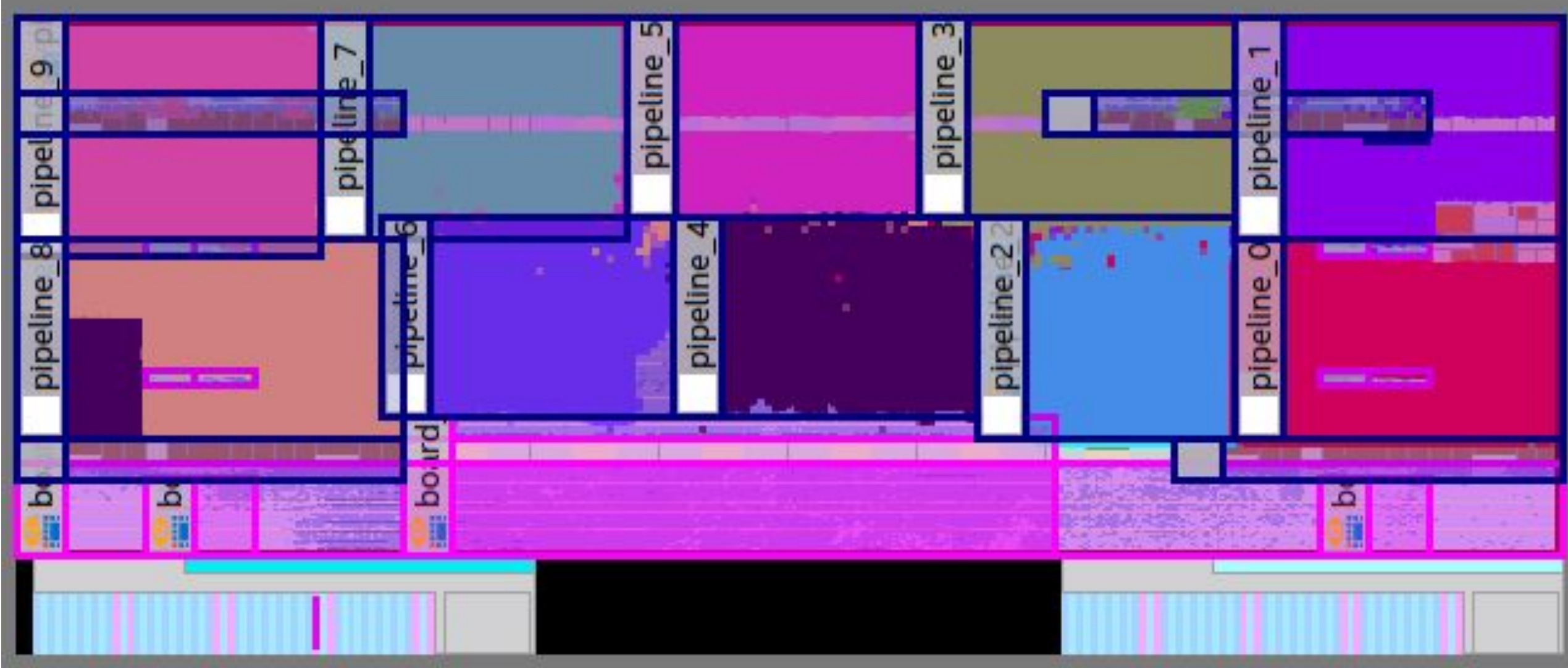






Terrible performance!

Adding logic lock regions

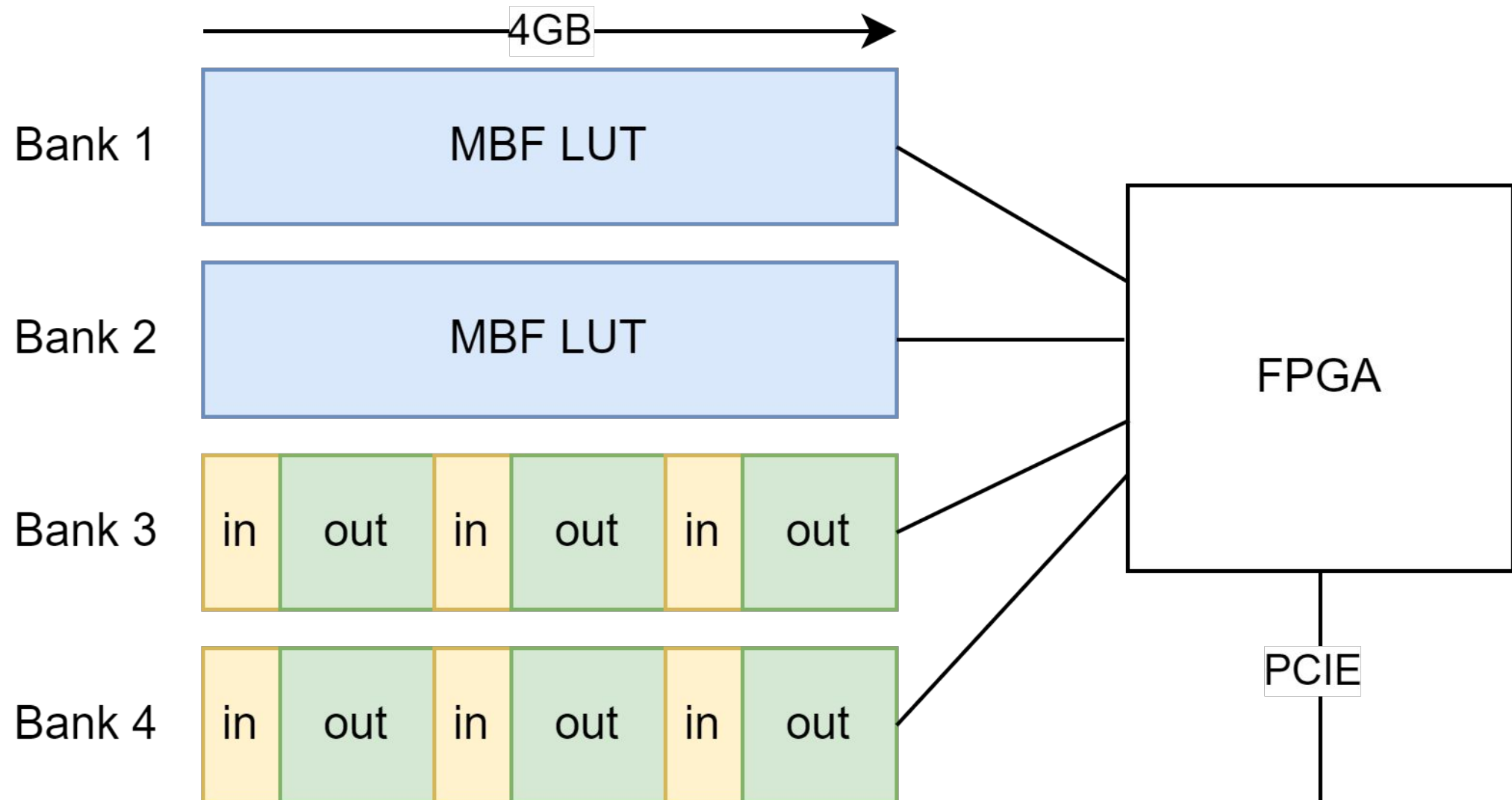


450 MHz!

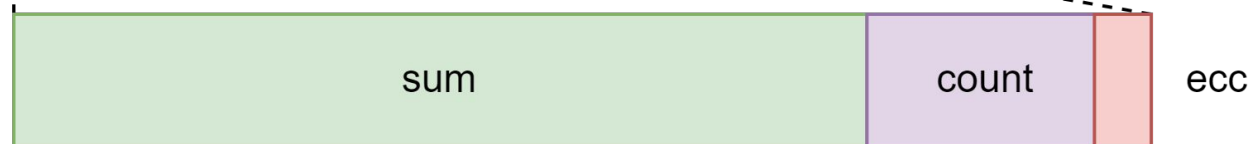
92% Logic Density

300 Mbits/s

FPGA Card Memory



Buffer Blocks



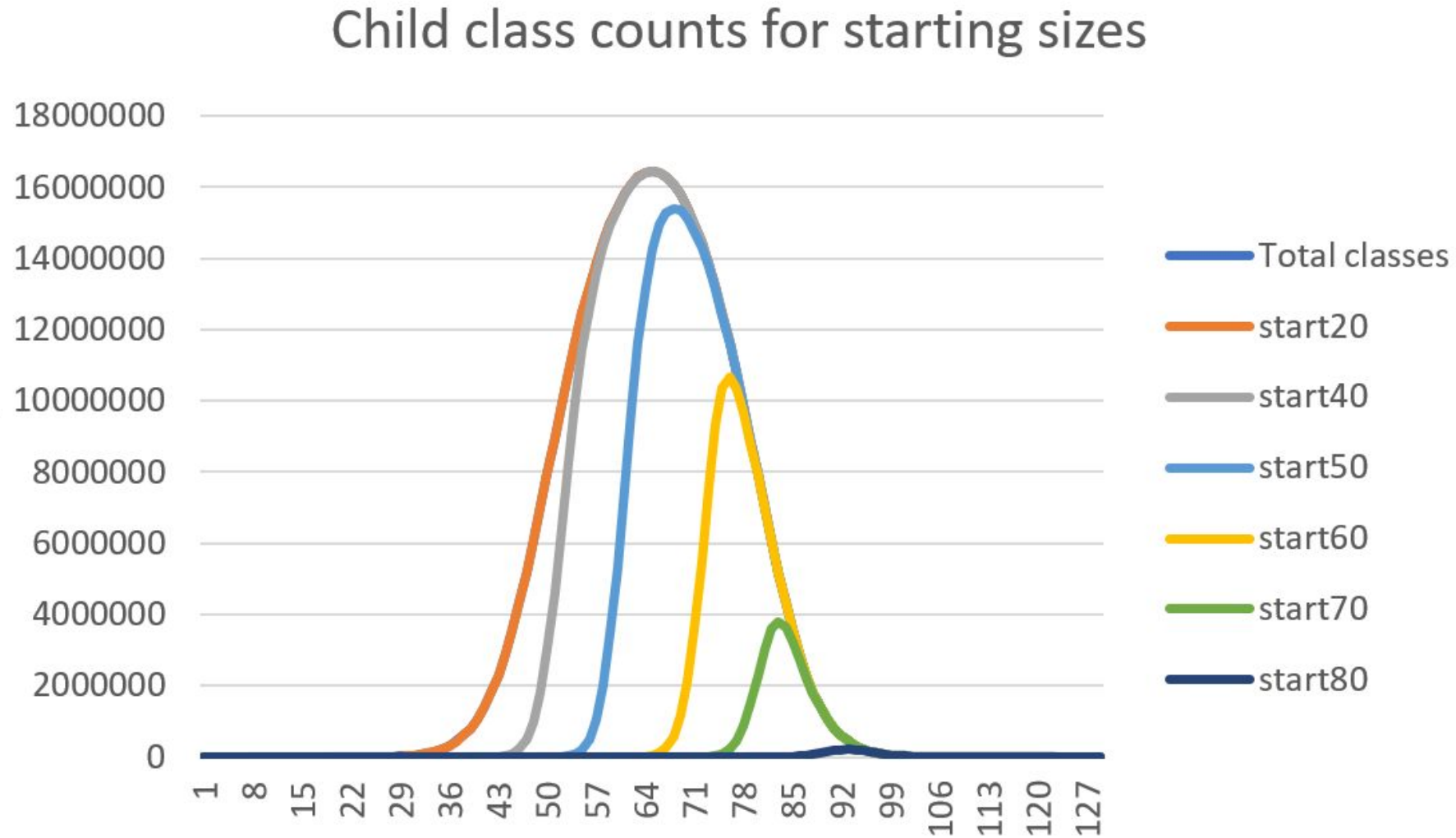
$$\sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

$$\sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 1$$

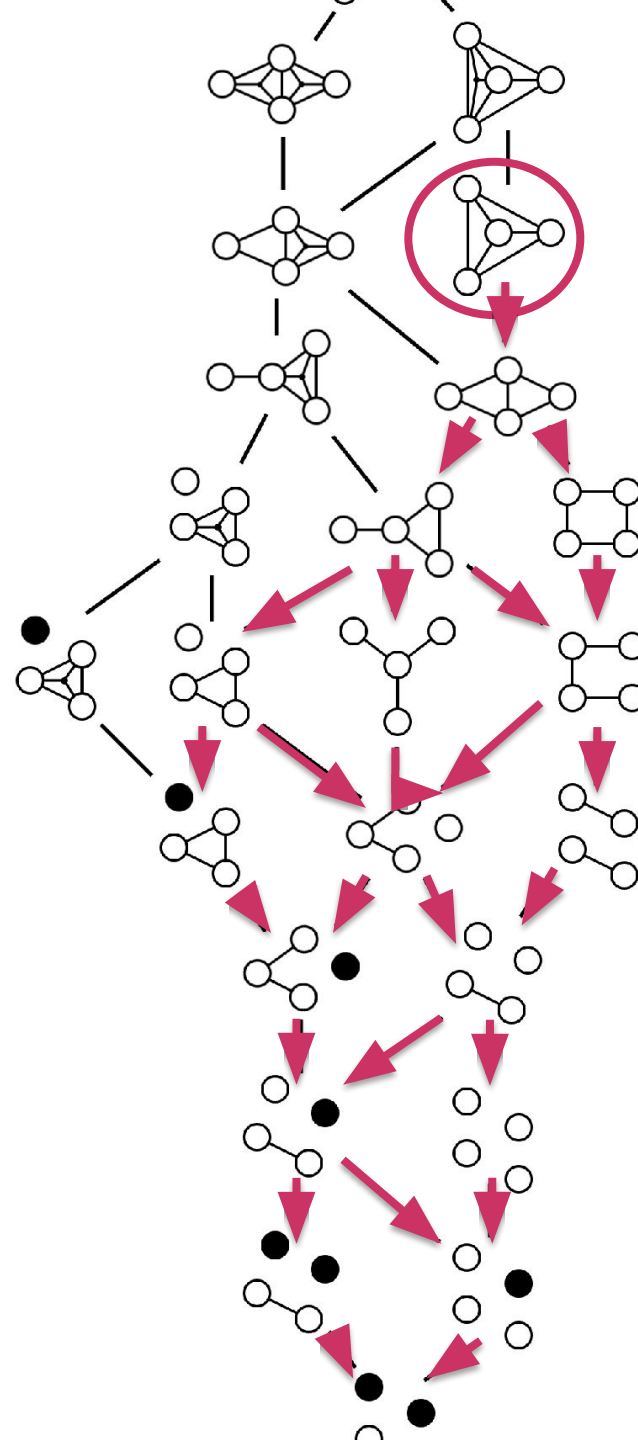
Supercomputing

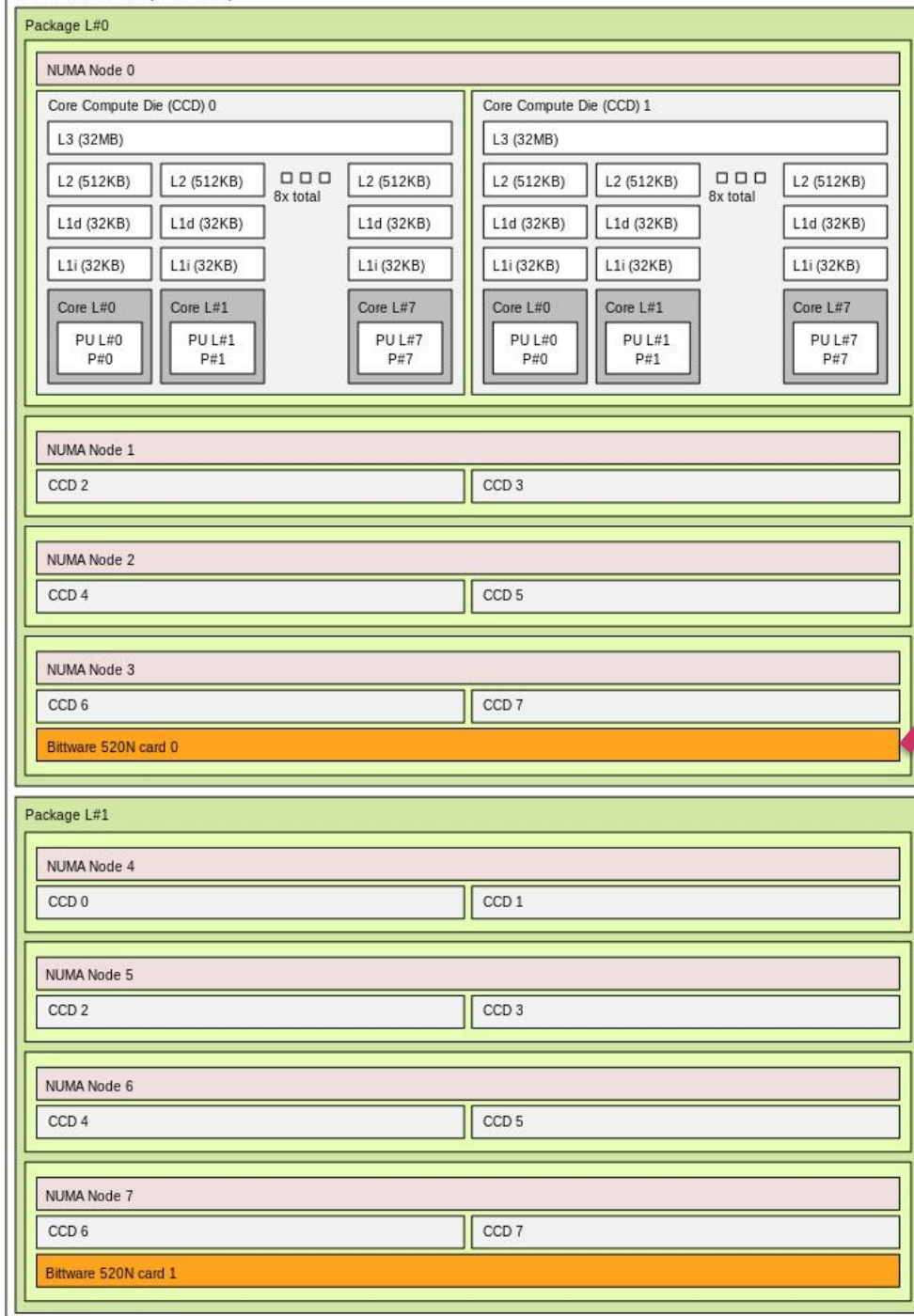
$$D(n+2) = \sum_{\alpha \in R_n} |[\perp, \alpha]| D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} |[\beta, \top]| \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

$$D(n+2) = \sum_{\alpha \in R_n} |[\perp, \alpha]| D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} |[\beta, \top]| \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$



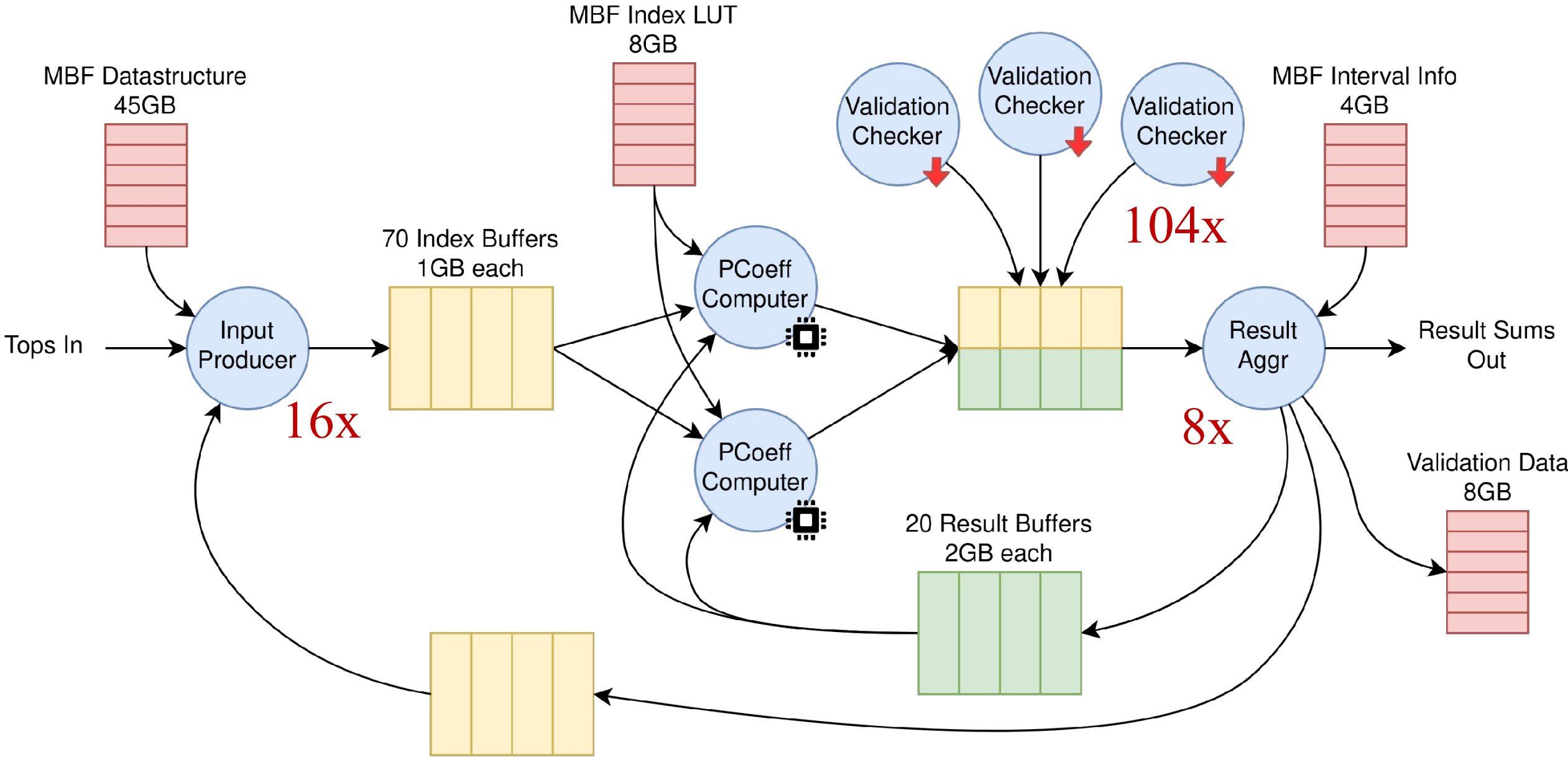
Beta Generation





Noctua 2 FPGA Node





$$D(n+2) = \sum_{\alpha \in R_n} |[\perp, \alpha]| D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} |[\beta, \top]| \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

$$D(n+2) = \sum_{\alpha \in R_n} |[\perp, \alpha]| D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} |[\beta, \top]| \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha, \gamma}}$$

5.8 α per second
490 million in total

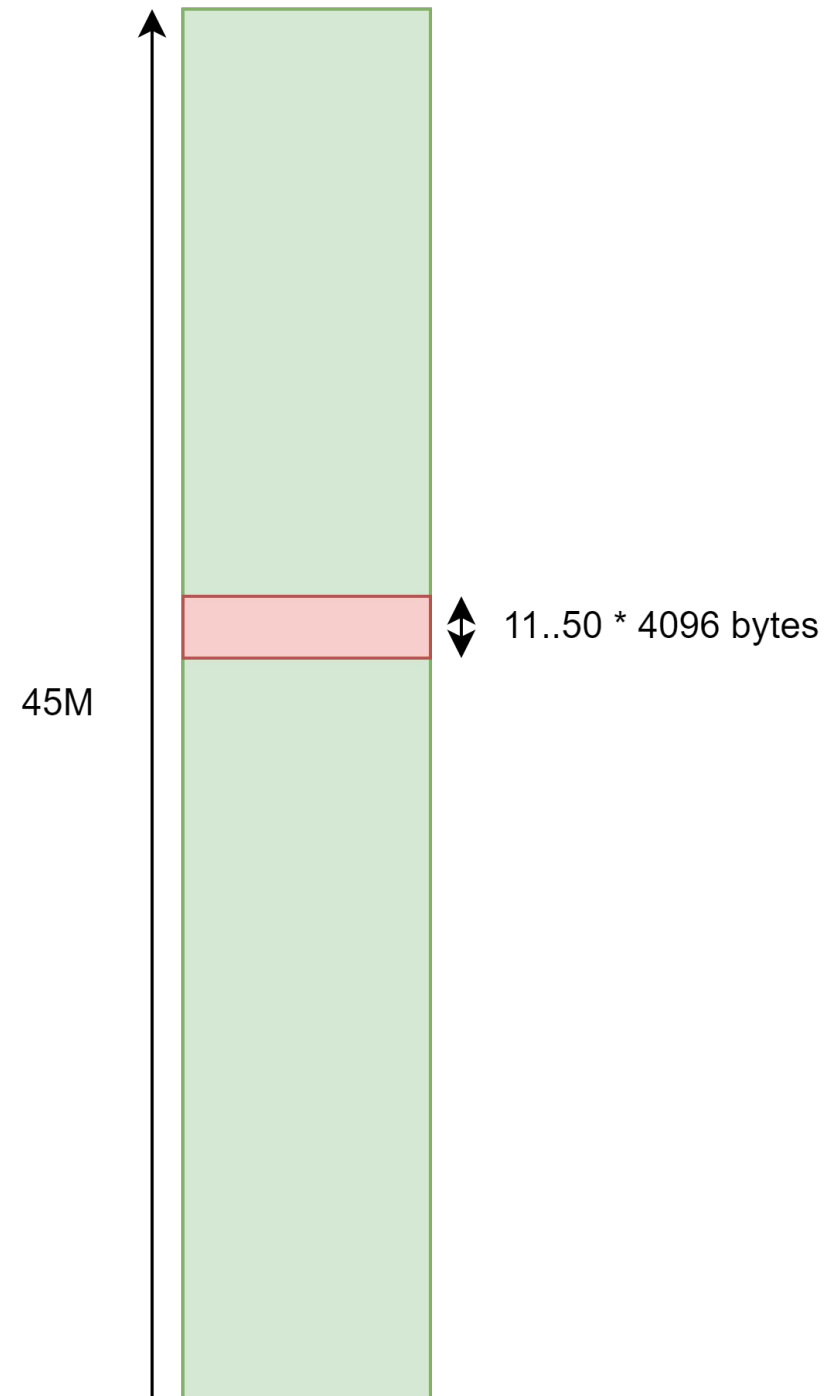
- 15'000 jobs
- 33'000 tops/job
- 100 mins / job
- 16 FPGA servers

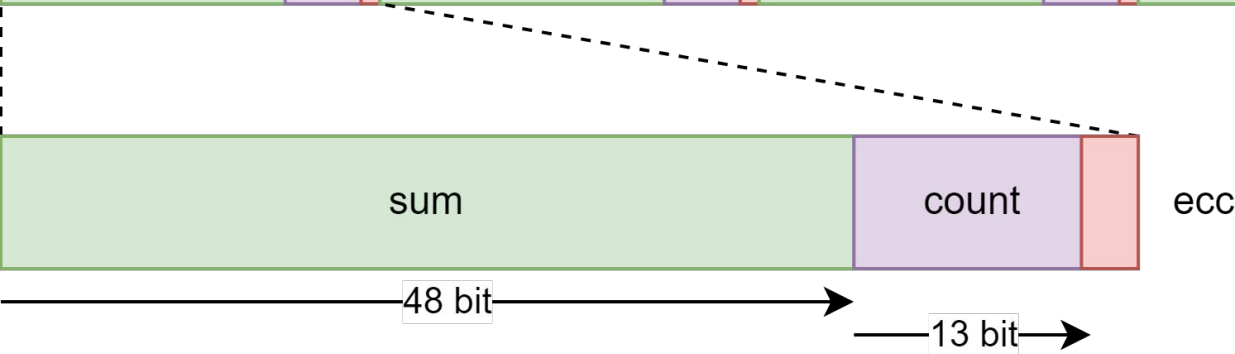
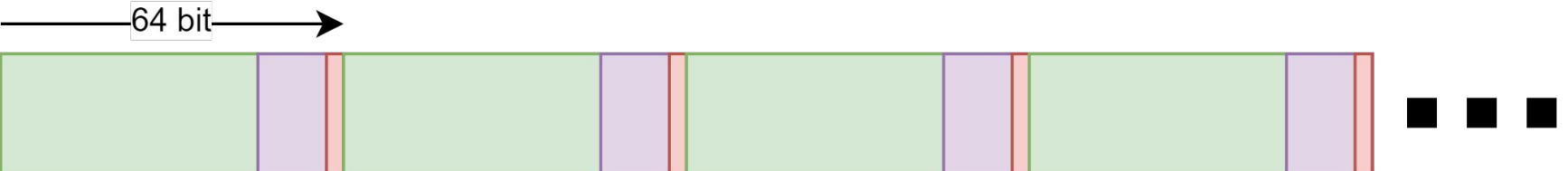
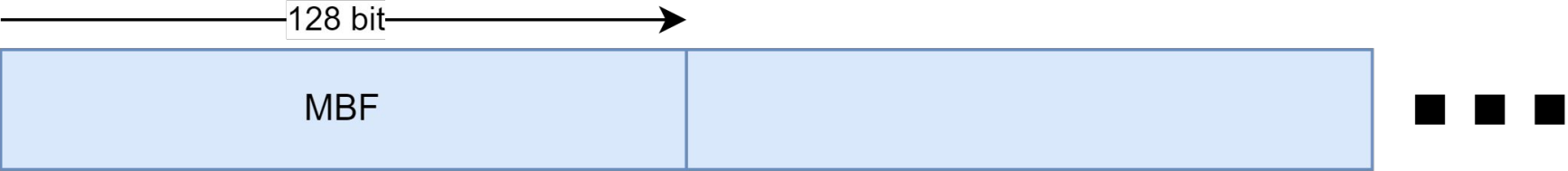
- After 4 months on Noctua 2
- 47'000 FPGA hours in total

March 8th 2023, at 5pm:

286386577668298411128469151667598498812366

- Large error blocks in result bufs
- Aligned to 4096 bytes
- “Forgot to copy”?
- Fixed now





$$\sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 2^{C_{\alpha,\gamma}}$$

$$\sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 1$$

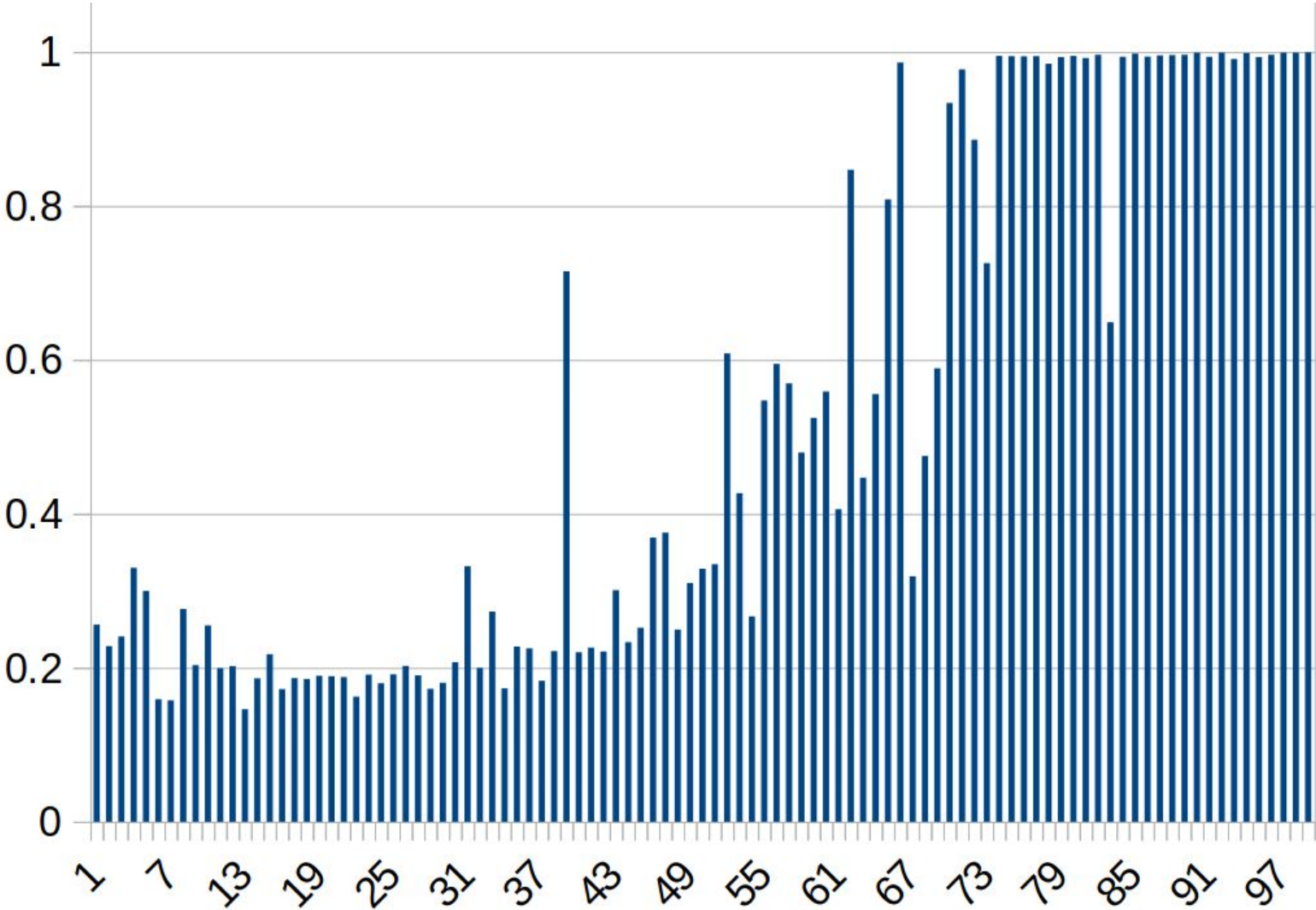
$$D(n + 1) = \sum_{\alpha \in R_n} D_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \simeq \beta : \alpha \leq \delta}} \frac{D_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \leq \gamma}} 1$$

Lucky Checksum!

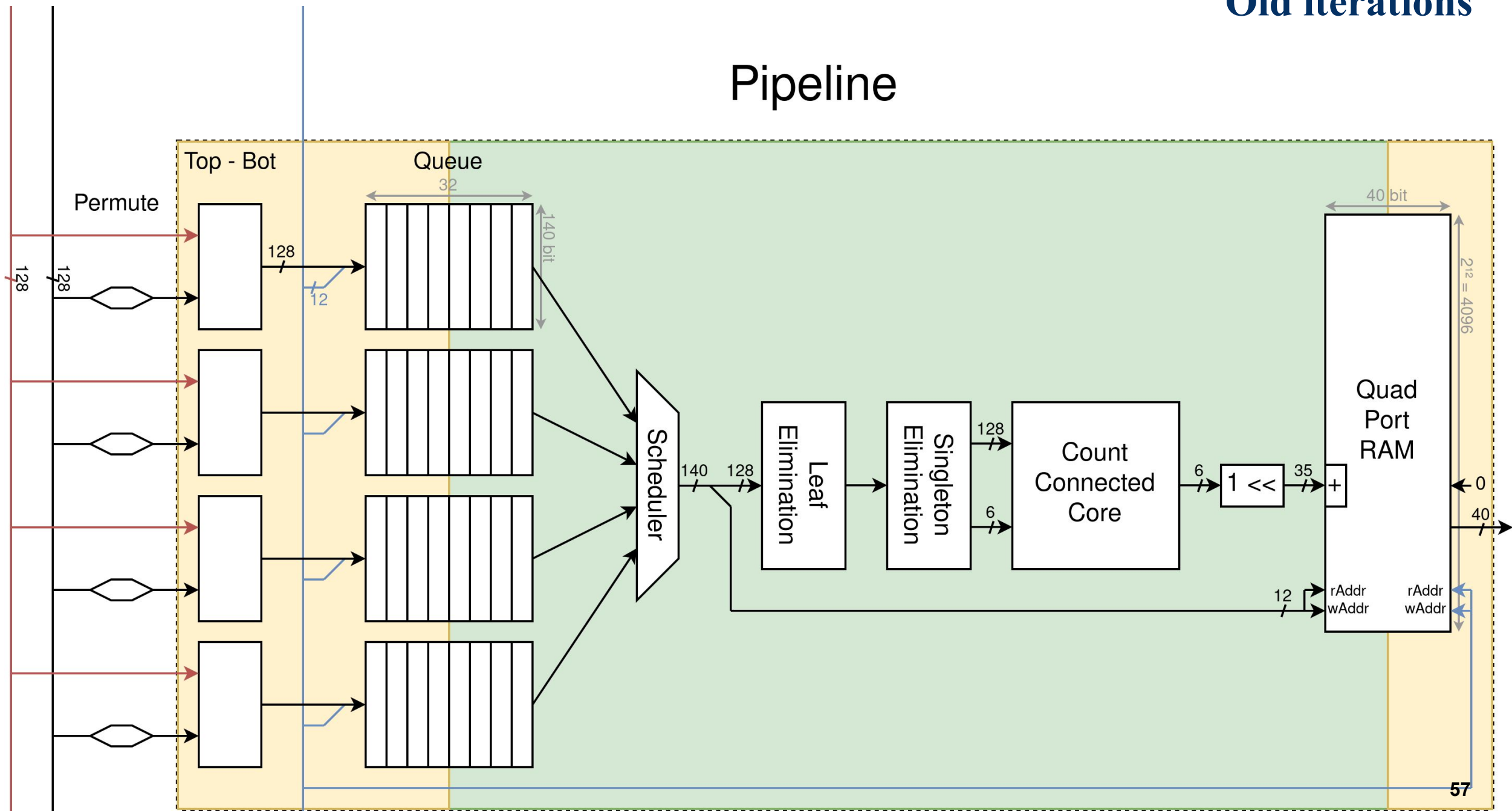


Appendix

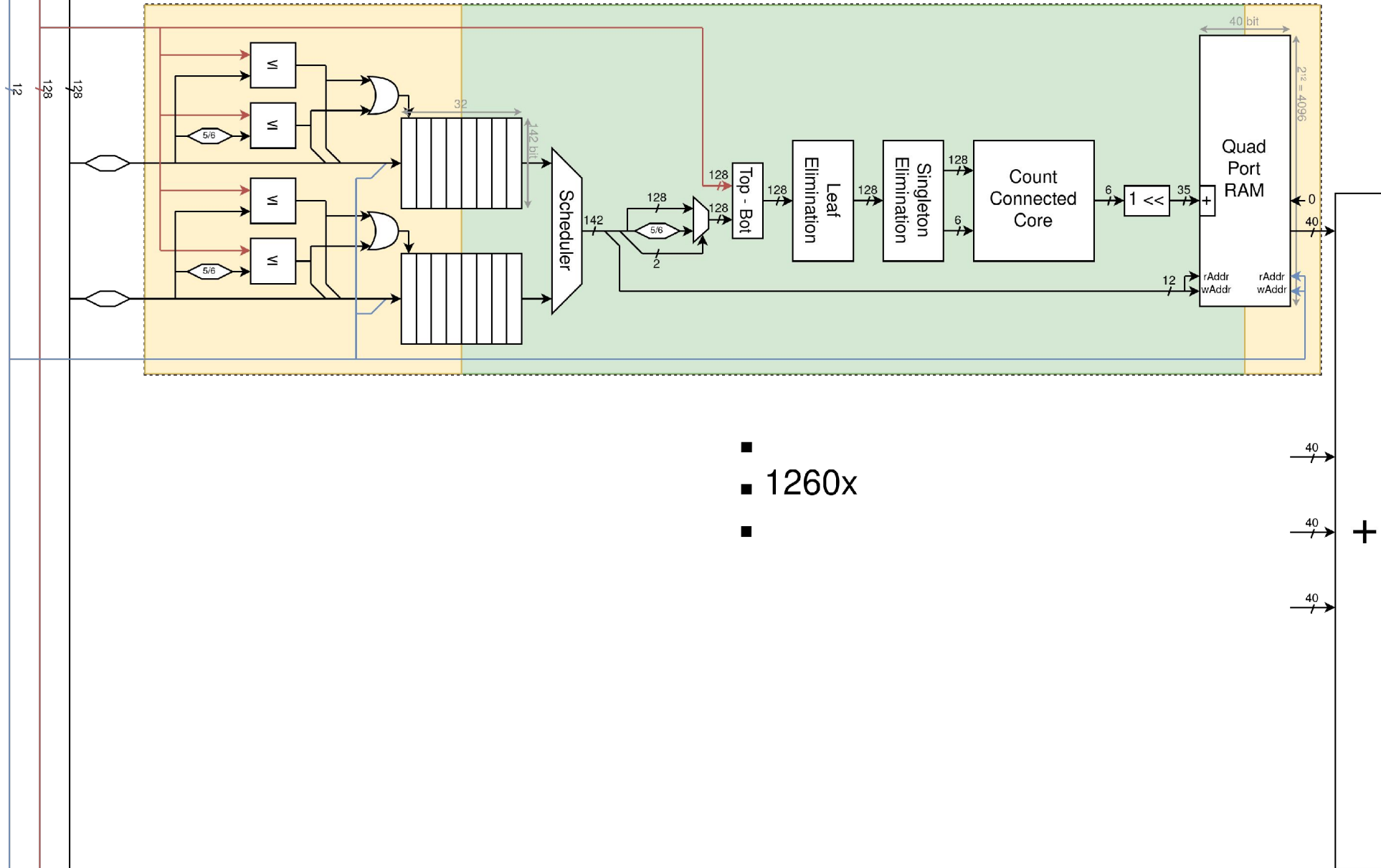
Efficiency Distribution



Pipeline



Pipeline



-
- 1260x
-

